

NumDuel_実装仕様書

0. 文書情報

項目	内容
文書名	Num Duel 実装仕様書
システム名	Num Duel
対象	ログイン済みユーザー向け 1 対 1 リアルタイム数字当て対戦
実装リポジトリ	<code>github.com/numduel/numduel</code>
デフォルトブランチ	<code>develop</code>
文書版	2.0（実装反映版・提出用）

1. ゲームルール1.1 秘密の数字

ルール 内容

使用する数字 0 ～ 9

桁数 4 桁固定

重複 不可

例 1234

1.2 ゲームの流れ

マッチングが成立し、対戦相手が決まる

両プレイヤーが秘密の数字を登録する（制限時間 SECRET_SETUP_SECONDS 秒）

プレイヤーは交互に相手の数字を予想する

予想のたびに各桁の判定結果が返る

先に 4 桁すべて一致したプレイヤーが勝利

1.3 判定方法

各桁について「正しい位置に正しい数字があるか」のみを判定する。

相手の秘密の数字 自分の予想 結果

1234 1456 × ○ × ×

表示 意味

- （当たり） その桁の数字が位置も含めて一致
 - ×（外れ） その桁は一致しない（数字が含まれていても位置が違えば外れ）
- API では 0 = 外れ、1 = 当たり。フロントエンドが ○ / × に変換する。

1.4 制限時間

項目	内容
秘密数字登録	SECRET_SETUP_SECONDS 秒（両者登録完了で開始、期限切れでゲーム終了・勝者なし）
1 ターン	TURN_DURATION_SECONDS 秒
ターン未入力時	ランダムな 4 桁（重複なし、crypto/rand 使用）を自動送信し、次ターンへ

1.5 勝利条件

4 桁すべて一致（4 つすべて当たり）したプレイヤーが勝利。

1.6 先手ルール

マッチング待機中のユーザーは登録順に対戦相手を決定する。先に待機キューへ登録したユーザーを player1（先手）、次を player2（後手）とする。

1.7 乱数生成

タイムアウト時の自動予想には crypto/rand を用いて重複なし 4 桁を生成する。予測可能な乱数はゲームの公平性を損なうため math/rand は使用しない。

2. 技術スタック2.1 アプリケーション

区分 採用

フロントエンド React + TypeScript + Vite

状態管理 React Context + useReducer（対戦画面のみ）

ルーティング React Router v6

バックエンド Go 1.25

HTTP Echo v4

WebSocket gorilla/websocket

ORM GORM

DB（本番） PostgreSQL 15
DB（バックアップ） PostgreSQL
DB（テスト） SQLite
Redis go-redis v9
認証 JWT（HttpOnly Cookie `access_token`）
リフレッシュトークン 不透明ランダム文字列（crypto/rand 64 バイト、hex エンコード）
パスワード bcrypt（cost = 12）
ID UUID v4
フロント FE テスト Vitest / Testing Library
バック テスト Go testing / testify

2.2 CI/CD / 運用

区分 採用
CI/CD GitHub Actions
コンテナ Docker / Docker Compose
レジストリ GHCR
シークレット GitHub Secrets
負荷試験 k6（同時対戦 100 ゲーム、P95 レイテンシ 500ms 以下を目標）

2.2 追記

区分	採用
フロント本番ホスティング	Vercel
CI/CD ワークフロー	<code>.github/workflows/cicd.yml</code>
負荷試験（CI）	k6 <code>/health</code> （ <code>k6/scenarios/health.js</code> ）

3. 設計原則3.1 データの役割分担

保存先 役割 データ
PostgreSQL 永続データ ユーザー、ゲーム、予想履歴、勝敗履歴、ランキング、各種ログ、マッチングキュー、リフレッシュトークン
Redis 揮発性の補助ストア ターン期限、連打防止ロック、JWT 失効、強制ログアウト時刻、WS 接続 ID、バックアップ状態

WebSocket リアルタイム配信 通知・同期イベント

補足

ゲーム状態の真実値は常に PostgreSQL とする。Redis のターン情報は期限管理用であり、喪失時は DB から復元する。

リフレッシュトークン本体は Cookie で運搬するが、照合用ハッシュとメタデータは PostgreSQL の refresh_tokens に保存する。

クライアントが保持する状態は表示用であり、サーバー正本と矛盾した場合はサーバーを優先する。

3.2 層構成

Router → Middleware → Controller → UseCase → Domain / Infrastructure

層 責務 主な構成要素

Router HTTP/WS の入口、Middleware 適用、Controller への振り分け Echo ルート、Middleware チェーン

Controller 入出力変換、認証情報の取得、UseCase 呼び出し、HTTP/WS レスpons生成 AuthController, GameController等

UseCase トランザクション管理、状態遷移、Redis/WS 連携、整合性保証 SubmitGuessUseCase 等

Domain エンティティ、値オブジェクト、ゲームルール、純粋関数 Game, User, JudgeDigits 等

Infrastructure Repository 実装、Redis、WebSocket Hub、外部 I/O postgres/, redis/, websocket/

3.3 層間依存ルール

ルール 内容

Domain の独立性 Domain は Infrastructure・UseCase・Controller に依存しない
状態変更の経路 ゲーム状態の変更は必ず UseCase を経由する

Controller の限定 Controller はリクエスト受付とレスポンス返却のみ行う

WS 受信 WebSocket 受信ハンドラから直接 DB を更新しない

WS 通知タイミング WebSocket 通知は DB コミット後のみ送信する

Repository 呼び出し Controller は Repository を直接呼び出さない

判定ロジックの配置 桁判定・勝敗判定は Domain の純粋関数のみで行う

3.4 各層の禁止事項

層 禁止すること

Domain DB ・ Redis ・ WebSocket へのアクセス、副作用、Infrastructure への依存
UseCase HTTP レスポンスの直接生成、ゲームルールの Domain 外実装
Controller 業務ロジック、DB/Redis 直接操作、Repository 直接呼び出し、ゲーム判定
Router UseCase 直接呼び出し、DB/Redis 直接操作、レスポンス組み立て

層 禁止すること

Infrastructure ゲームルール判定、ターン進行決定、Controller の呼び出し

3.5 桁判定（Domain 純粋関数）

桁判定・勝敗判定は副作用のない純粋関数として Domain 層に実装する。

```
func JudgeDigits(secret, guess [4]int) [4]DigitResult
```

```
func IsWin(results [4]DigitResult) bool
```

```
secret guess results hitCount
```

```
1234 1234 1,1,1,1 4
```

```
1234 1456 0,1,0,0 1
```

```
1234 5678 0,0,0,0 0
```

関数 入力 出力 説明

JudgeDigits 秘密数字、予想数字 [4]DigitResult 各桁を比較し 0= 外れ / 1= 当たり

IsWin 判定結果 bool 4 桁すべて 1 なら true

配置ルール

Controller ・ WebSocket ハンドラ ・ Infrastructure 内で判定を行わない。

判定関数内から DB ・ Redis ・ WebSocket を呼び出さない。

相手の秘密数字をログ ・ API レスポンス ・ WS ペイロードに含めない。

3.6 同時実行制御の原則

ゲーム単位の更新は必ず SELECT ... FOR UPDATE で games 行をロックする。

項目 内容

ロック単位 1 ゲーム = 1 行 (games.id)

ロック方式 SELECT ... FOR UPDATE

適用 UseCase SetSecretNumber, StartGame, SubmitGuess, HandleTimeout, FinishGame, CancelGameBySecretTimeout 等

タイムアウト処理と手動予想が競合した場合、先にロックを取得した処理のみが進行する。

後着の処理 結果

手動 SubmitGuessUseCase not_your_turn または
game_not_started / game_already_finished

HandleTimeoutUseCase ゲームが既に進行済みなら何もしない（正常終了）

ゲーム状態 返却エラーコード

WAITING_SECRET game_not_started

FINISHED game_already_finished

自ターンでない not_your_turn

3.6.1 マッチングの同期実行

原則 内容

実行タイミング POST /api/matching/start 処理中のみペアリング

TX queue INSERT MatchPlayersUseCase を 同一 TX

原則 内容

Worker 実装しない

通知 StartMatchingUseCase が COMMIT 後 に MATCHED

2 人目 後から start したユーザーでペアリング成立

3.7 トランザクション原則

原則 内容

開始・コミット DB 更新は UseCase 内のトランザクションで行う

ロック順序 ゲーム更新前に games 行を FOR UPDATE で取得する

コミット前通知禁止 DB コミット前に WebSocket 通知を送らない

勝利処理 予想による勝利確定・match_histories 作成・win_count 加算は

SubmitGuessUseCase の同一トランザク

ション内で完結する

ロールバック DB 更新失敗時は変更を保存せず、Redis 更新・WS 通知も行わない

3.7.1.1 問題と原則

問題 内容

部分 COMMIT 勝利判定後に guesses のみ COMMIT し、match_histories /
win_count が未更新になる

別 TX 呼び出し FinishGameService を独立トランザクションで呼ぶと、前半だけ DB

に残る

判定と書き込みの混在 勝利判定処理が単独で DB を更新すると、エラー時に不整合が残る

原則

原則 内容

1 判定は Domain のみ JudgeDigits / IsWin は純粋関数。DB ・ Redis ・ WS に触れない

2 書き込みは 1 TX guesses INSERT 〜勝利終了処理（または次ターン UPDATE）まで同一トランザクション

3 COMMIT は 1 回 すべて成功した場合のみ COMMIT 。途中 error は ROLLBACK

4 FinishGame は内部サービス FinishGameService は tx を受け取るのみ。自前 Begin / Commit 禁止

5 副作用は COMMIT 後 Redis 更新 ・ WS 通知 ・ activity_logs は COMMIT 成功後のみ

3.7.1.2 層ごとの責務

層 勝利関連の責務 DB アクセス

Domain IsWin。JudgeDigits は単体テスト ・ Verify 一致検証用 なし

Infrastructure SecretHashService.Hash / Verify なし

SubmitGuessUseCaseTX 開始 〜 Verify 〜 IsWin 〜書き込み 〜 COMMIT / ROLLBACKあり

HandleTimeoutUseCaseSubmitGuess と同一 TX パターン あり

FinishGameService 勝利時 games / match_histories / win_count 更新 呼び出し元 tx のみ

Controller / WS UseCase 呼び出しのみ なし

3.7.1.3 同一 TX に含める DB 操作

操作 未勝利 勝利

INSERT guesses ○ ○

UPDATE games （次ターン） ○ —

UPDATE games （ FINISHED ） — ○ （ FinishGameService 内）

操作 未勝利 勝利

INSERT match_histories — ○

UPDATE users.win_count — ○

いずれか1件でも失敗した場合：上記すべて ROLLBACK（guesses も含め DBに残さない）。

3.7.1.4 SubmitGuessUseCase / HandleTimeoutUseCase の TX 境界

【TX 外】Redis キー game:{gameId}:player:{playerId}:guess_lock を Lua で acquireguessNumber バリデーション（NewGuessNumber）

【TX 内】BEGINSELECT games ... FOR UPDATE参加者・ターン・status 検証 → 失敗時 ROLLBACKopponentSecretHash ← games から取得digitResults ← SecretHashService.Verify(opponentSecretHash, guessNumber, gameId, opponentPlayerSlot)isWin ← Domain.IsWin(digitResults)INSERT guessesisWin == true → FinishGameService(ctx, tx, game, winnerIDisWin == false → UPDATE games（次ターン）いずれか error → ROLLBACKCOMMIT

【COMMIT 成功後のみ】Redis ターン更新 / 削除GUESS_RESULT / GAME_OVER / TURN_CHANGED 送信activity_logs 保存

3.7.1.5 FinishGameService の位置づけ

項目 内容

名称 5 章 FinishGameUseCase 節で定義する FinishGameService（HTTP エンドポイントなし）

呼び出し元 SubmitGuessUseCase / HandleTimeoutUseCase のみ

TX 呼び出し元から渡された tx を使用。自前で Begin / Commit / Rollback しない
失敗 error を返す → 呼び出し元が ROLLBACK

通知 GAME_OVER / activity_logs は呼び出し元が COMMIT 後に実行

```
func FinishGameService(  
ctx context.Context,  
tx Transaction,  
game Game,  
winnerID uuid.UUID,  
) error
```

3.7.1.6 禁止事項

禁止 理由

FinishGameService を独立 TX で呼ぶ 部分 COMMIT で不整合

guesses INSERT 後に別 goroutine で FinishGameエラー時に取り消せない

win_count を TX 外で更新 ランキングと実勝敗がずれる
COMMIT 前に GAME_OVER 送信 ROLLBACK 後もクライアントが勝利表示

禁止 理由

Controller / WS で勝利判定 Domain 外判定 (3.5 違反)

3.8 WebSocket 通知原則

原則 内容

保存先 WS は通知・同期専用。ゲーム状態の PostgreSQL

送信タイミング DB コミット成功後のみ Server → Client イベントを送信する

秘密情報 相手の秘密数字・相手の予想内容を WS で送らない

再接続 切断後は SYNC_REQUEST で DB から状態を再同期する

認証 接続後 5 秒以内の AUTH 必須。未認証イベントは拒否する

3.9 認証検証の原則

項目 内容

アクセストークン JWT HttpOnly Cookie (`access_token`)。Authorization ヘッダーは使
用しない)

JWT Claims sub, role, jti, iat, exp

検証順 署名 → exp → jwt:revoked:{jti} → iat < force_logout_before

期限切れ exp 超過のみ → token_expired (HTTP 404)。クライアントは
/api/auth/refresh で更新

その他失効 署名不正・失効・強制ログアウト → unauthorized (HTTP 401)

リフレッシュ Cookie refresh_token から取得。Request Body 不使用

3.10 秘密数字と予想処理3.10.1 全体フロー

マッチング成立 (status = WAITING_SECRET) ↓ 両者 SET_SECRET (UseCase で
Hash → games.player*_secret にハッシュ保存) ↓ StartGameUseCase (status =
IN_PROGRESS、player1 先手) ↓ 【交互ターン】 手番 → GUESS (WS 平文 guess
+ DB 相手ハッシュを UseCase で照合) ↓ Verify → IsWin → guesses INSERT → 未
勝利 : 次ターン / 勝利 : FinishGameService ↓ COMMIT 後 → GUESS_RESULT →
TURN_CHANGED または GAME_OVER ↓ 4 桁一致 → FINISHED (guess_win)
フェーズ status プレイヤー操作 サーバー UseCase
秘密数字登録 WAITING_SECRET SET_SECRET SetSecretNumberUseCase

ゲーム開始 WAITING_SECRET → IN_PROGRESS — StartGameUseCase（両者登録完了時）

対戦 IN_PROGRESS GUESS SubmitGuessUseCase

ターン未入力 IN_PROGRESS —（自動） HandleTimeoutUseCase

秘密数字期限切れ WAITING_SECRET → FINISHED —
CancelGameBySecretTimeoutUseCase

3.10.2 秘密数字の登録3.10.2.1 ルール

項目 内容

桁数 4 桁固定

使用数字 0 ～ 9

重複 不可

登録回数 1 プレイヤー 1 ゲーム 1 回のみ

登録可能 status WAITING_SECRET のみ

期限 created_at + SECRET_SETUP_SECONDS（両者未登録のまま期限切れ → 勝者なし終了）

3.10.2.2 クライアント → サーバー

項目 内容

イベント WS SET_SECRET

UseCase SetSecretNumberUseCase

ペイロード { type, gameId, secretNumber } — 平文 4 桁。送信はこの 1 回のみ

禁止 クライアントでハッシュ化して送らない。サーバーが正本

3.10.2.3 サーバー処理 — SetSecretNumberUseCase

順 処理 層

1 Redis キー game:{gameId}:player:{playerId}:secret_lock を Lua で
acquireInfrastructure

2 NewSecretNumber で形式検証 Domain

3 SecretHashService.Hash(secretNumber, gameId, playerSlot) Infrastructure

4 平文 secretNumber は以降の処理で参照しない。スコープ外へ持ち出さない
UseCase

5 BEGIN → games FOR UPDATE UseCase

6 参加者・status・未登録を検証 UseCase

7 player1_secret または player2_secret に ハッシュ文字列 を保存 DB
8 COMMIT UseCase
9 両者登録済みなら StartGameUseCase UseCase
10 COMMIT 後 : 片方のみ登録 → WS 送信なし WS
ハッシュ方式
項目 内容
方式 位置別 HMACSHA256。桁ごと判定に必要。単一 bcrypt ダイジェストでは桁判定不可
入力 secretNumber 4 桁、gameID、playerSlot 1 / 2、サーバー側 pepper
pepper 環境変数 GAME_SECRET_PEPPER
1 桁あたり HMAC-SHA256(pepper, "{gameID}:{playerSlot}:{position}:{digit}") → hex
DB 保存形式 4 ダイジェストを : 連結した 1 文字列。例
: a1b2...:c3d4...:e5f6...:7890...
平文禁止 PostgreSQL ・ バックアップ DB ・ ログに平文 4 桁を保存しない
DB 保存
カラム 内容
games.player1_secretplayer1 登録 secret の 位置別 HMAC 連結文字列。
VARCHAR(255)
games.player2_secretplayer2 同上

Entity / Domain: Game.Player1Secret / Player2Secret の *SecretNumber 平文型は DB 永続化に使わない。Repository はカラムを string ハッシュとしてマッピングする。

3.10.2.4 非開示ルール — 秘密数字

公開先 自分の secret 平文 相手の secret ハッシュ DB 値
自分 UI 入力値として保持可。サーバーは返さない 見えない 見えない
相手 WS / HTTP 送らない 送らない 送らない
GameStateDTO 含めない 含めない 含めない
activity_logs / CSV含めない 含めない 含めない
バックアップ DB平文含まない 平文含まない 含む。本番 DB 同等の取り扱い

3.10.3 相手が予想したときの処理3.10.3.1 前提

項目 内容

イベント WS GUESS

UseCase SubmitGuessUseCase 手動 / HandleTimeoutUseCase 自動、isAuto=true

照合入力 WS で受信した平文 guessNumber と、DB の 相手 secret ハッシュ

照合実行 UseCase が SecretHashService.Verify を呼び出し。WS ハンドラ・クライアントでは行わない

勝敗 Domain.IsWin(digitResults)。純粋関数。DB 書き込みなし

DB 反映 同一 TX 内。 §3.7.1

照合対象の決定

予想者 userID DB から読むカラム

player1_id player2_secret — ハッシュ文字列

player2_id player1_secret — ハッシュ文字列

3.10.3.2 処理フロー — SubmitGuessUseCase

【TX 外】

guess_lock 取得

guessNumber バリデーション (NewGuessNumber)

【TX 内】

BEGIN

SELECT games ... FOR UPDATE

status = IN_PROGRESS / 参加者 / 自ターン を検証

opponentSecretHash ← games から取得

digitResults ← SecretHashService.Verify(opponentSecretHash, guessNumber, gameId, opponentPlayerSlot)

hitCount ← digitResults の 1 の個数

isWin ← Domain.IsWin(digitResults)

INSERT guesses (guess_number, digit_results, hit_count, is_auto, turn)

isWin == true → FinishGameService(tx, game, userID)

isWin == false → UPDATE games (current_turn 1、手番を相手へ)

COMMIT (いずれか失敗 → ROLLBACK。guesses も取り消し)

【COMMIT 後】

GUESS_RESULT → 両プレイヤー

isWin == true → GAME_OVER (reason: guess_win)

isWin == false → TURN_CHANGED Redis 次ターン

activity_logs (guess / timeout)

SecretHashService.Verify の契約

項目 内容

入力 保存済みハッシュ文字列、guessNumber、gameID、相手 playerSlot

処理 各 position 0..3 について、guess の桁で HMAC を再計算し保存ダイジェストと定数時間比較

出力 [4]DigitResult。1 = 当たり、0 = 外れ

配置 Infrastructure。Domain の JudgeDigits と 同一の結果 になること

HandleTimeoutUseCase も 同一パターン。自動予想数字は

RandomNumberService crypto/rand で生成する。

3.10.3.3 判定例

平文 secret はサーバー内部の説明用。DB には存在しない。

相手 secret 内部 予想 guess digitResults hitCount isWin

1234 1234 1,1,1,1 4 true

1234 1456 0,1,0,0 1 false

1234 5678 0,0,0,0 0 false

3.10.3.4 DB 書き込み

§3.10.3.4 変更なし。guesses.guess_number は従来どおり平文 4 桁 — 自分の履歴表示用。相手の secret ハッシュはguesses に含めない。

3.10.3.4 DB 書き込み（1回の GUESS あたり）

未勝利時（同一 TX）

テーブル 操作 内容

guesses INSERT guess_number, digit_results, hit_count, turn, is_auto

games UPDATE current_turn 1, current_turn_player_id = 相手

勝利時（同一 TX）

テーブル 操作 内容

guesses INSERT 同上

games UPDATE status=FINISHED, winner_id, finished_at (FinishGameService)

match_historiesINSERT 勝敗履歴 (FinishGameService)

users UPDATE 勝者 win_count 1 (FinishGameService)

エラー時: 上記すべて ROLLBACK。部分更新を DB に残さない (3.7.1)。

3.10.4 クライアントへの通知（予想後）

3.10.4.1 GUESS_RESULT（両プレイヤーへ）

COMMIT 成功後、参加者両者に送信する。

項目 含める 含めない

guessId ○ —

playerId（誰が予想したか） ○ —

項目 含める 含めない

digitResults ○ —

hitCount ○ —

isWin ○ —

isAuto ○ —

nextTurnPlayerID ○（勝利時は空文字） —

guessNumber（予想 4 桁そのもの） — ×

相手の secret — ×

```
{  
  "type": "GUESS_RESULT",  
  "data": {  
    "guessId": "uuid",  
    "playerId": "uuid",  
    "digitResults": 0, 1, 0, 0,  
    "hitCount": 1,  
    "isWin": false,  
    "isAuto": false,  
    "nextTurnPlayerID": "uuid"  
  }  
}
```

3.10.4.2 続くイベント

条件 送信順

isWin = false GUESS_RESULT → TURN_CHANGED

isWin = true GUESS_RESULT → GAME_OVER（reason: guess_win）

3.10.4.3 各プレイヤーが見える情報（対戦中）

情報 自分 相手

自分の secret UI 上のみ。サーバーは平文を保持・返却しない 見えない

相手の secret 見えない 見えない

自分の予想履歴（数字 + 結果） GameStateDTO.myGuesses / 画面 見えない（数字は非公開）

相手の予想回数 opponentGuessCount opponentGuessCount

直近 GUESS の digitResults GUESS_RESULT で双方受信 同上

相手が入力した guessNumber自分の履歴としてのみ 非公開

フロント禁止: 秘密数字の照合・勝敗判定をクライアントで行わない。

3.10.5 具体シナリオ

設定: player1 = 1234、 player2 = 5678（説明用平文。DB はハッシュのみ）

ターン 手番 操作 サーバー内部 通知

— 両者 SET_SECRET Hash → DB 保存 GAME_STATE_SYNC → TURN_CHANGED

1 player1 GUESS 1456 Verify(5678 ハッシュ , 1456) → 0,1,0,0 GUESS_RESULT
→ TURN_CHANGED

ターン 手番 操作 サーバー内部 通知

2 player2 GUESS 1234 Verify(1234 ハッシュ , 1234) → 1,1,1,1 GUESS_RESULT →
GAME_OVER

3.10.6 シーケンス — 手動 GUESS

PlayerA (手番) WS Handler SubmitGuessUseCase SecretHashService Domain
PostgreSQL

```
||||| |
| GUESS ( 平文 ) |||||
|| Execute — |||
|| BEGIN — |||
|| SELECT FOR UPDATE ————— |
|| read opponentSecretHash ————— |
|| Verify(guess) |||
|| digitResults |||
|| IsWin — |||
```

```

||| INSERT guesses ———- ||
||| FinishGame or next turn - ||
||| COMMIT ———- ||
| GUESS_RESULT ||||
| TURN_CHANGED/GAME_OVER |||

```

3.10.7 関連 UseCase 早見表

処理 UseCase WS (Client → Server) WS (Server → Client)

秘密数字登録 SetSecretNumberUseCase SET_SECRET 片方のみ: なし / 両者完了:
GAME_STATE_SYNC, TURN_CHANGED

予想 (手動) SubmitGuessUseCase GUESS GUESS_RESULT, TURN_CHANGED or
GAME_OVER

予想 (自動) HandleTimeoutUseCase — 同上

状態同期 SyncGameStateUseCase SYNC_REQUEST GAME_STATE_SYNC

勝利終了 DB FinishGameService (内部) — —

3.10.8 禁止事項

禁止 内容

DB 平文保存 player*_secret に平文 4 桁を保存しない

クライアント照合 秘密数字・桁判定をフロントで行わない

ハッシュの外部送信 secret ハッシュを WS / HTTP / ログ / CSV に含めない

Domain 外の IsWinIsWin 以外の勝敗判定を UseCase 外で行わない

復号依存 secret の 復号 に依存する設計にしない。本章はハッシュ照合

3.10.9 テストで確認することシナリオ 期待

1 SET_SECRET 後 DB player*_secret が平文 4 桁 ではない。 HMAC 連結形式

2 player1 が GUESS 照合対象は player2_secret ハッシュ

シナリオ 期待

3 SecretHashService.Verify JudgeDigits と同一 digitResults

4 GUESS_RESULT ペイロード 平文 secret ・ハッシュ ・ guessNumber を含めない

5 GameStateDTO player*_secret / ハッシュを含まない

6 勝利 GUESS で FinishGame 失敗 guesses 含め ROLLBACK

7 SET_SECRET 片方のみ Server → Client イベントなし

要点

項目 内容

秘密数字登録 WS 平文 → UseCase Hash → DB に ハッシュのみ。平文・ハッシュとも外部に出さない

予想照合 WS 平文 guess + DB ハッシュ → UseCase 内 Verify → IsWin → 1 TX

通知 COMMIT 後 GUESS_RESULT → TURN_CHANGED または GAME_OVER

4. ドメインモデル4.1 値オブジェクト

型 検証 生成

SecretNumber 4 桁・09・重複なし NewSecretNumber([4]int)

GuessNumber 同上 NewGuessNumber([4]int)

GameStatus WAITING_SECRET / IN_PROGRESS / FINISHED 定数

Role user / master 定数

DigitResult 0 / 1 定数

RefreshTokenStatusactive / revoked 定数

4.2 Entity: User

属性 型 説明

ID uuid.UUID PK

Username string UNIQUE、3～50文字、英数字と_

Email string UNIQUE、RFC5322 準拠

PasswordHash string bcrypt cost 12

Role Role user / master

WinCount int 累計勝利数

DeletedAt *time.Time* 削除

DeletedBy uuid.UUID 削除した管理者

LastActivityAt *time.Time* 最終操作

CreatedAt / UpdatedAt*time.Time* —

メソッド: IsDeleted(), IsMaster(), CanMatch()

4.3 Entity: Game (集約ルート)

属性 型 説明

ID uuid.UUID PK

Status GameStatus —

Player1ID uuid.UUID 先手

Player2ID uuid.UUID 後手

属性 型 説明

Player1Secret Player1SecretHash string。空 = 未登録 —

Player2Secret Player2SecretHash string —

CurrentTurnPlayerID *uuid.UUID* —

CurrentTurn int 1 始まり

WinnerID uuid.UUID —

StartedAt time.Time —

FinishedAt time.Time —

CreatedAt / UpdatedAt time.Time —

平文を Entity に保持 ハッシュ文字列を SetSecretHash 等で設定 —

メソッド : IsParticipant, IsCurrentTurn, CanGuess, SetSecret, BothSecretsSet,

AddGuess, Finish, CancelBySecretTimeout

予想履歴は guesses テーブルに保持。追加は必ず Game 経由。

4.4 MatchHistory（読み取り専用）

勝敗が確定したゲーム終了時のみ FinishGameUseCase が作成する。

secret_setup_timeout では作成しない。

4.5 Ranking（Read Model）

RebuildRankingUseCase が users.win_count から rankings を上書きする。削除済みユーザー・master は除外。

5. UseCase5.1 一覧

認証・ユーザー

UseCase 概要

RegisterUserUseCase users INSERT

LoginUseCase 認証。JWT + refresh 発行・DB 保存 + Set-Cookie + login_logs

RefreshTokenUseCase リフレッシュトークンを検証し、新しいアクセストークンとリ

フレッシュトークンを発行する
LogoutUseCase JWT 失効・WS セッション削除
GetMeUseCase 自分の情報
AutoLogoutUseCase SESSION_TIMEOUT_MINUTES 分無操作ユーザーを強制ログアウト
StartMatchingUseCase キュー登録 + 同一 TX 内即時ペアリング
MatchPlayersUseCase 内部用。StartMatching からのみ。先頭 2 人で Game 作成
WebSocket
UseCase 概要
AuthenticateWebSocketUseCase WS 接続の JWT 検証・接続登録
マッチング
UseCase 概要
StartMatchingUseCase キュー登録・即時マッチング試行
CancelMatchingUseCase キュー削除
GetMatchingStatusUseCase 待機状態取得
MatchPlayersUseCase 先に待機したユーザーから順番にペアリング・ゲーム作成
ゲーム

UseCase 概要
GetGameStateUseCase ゲーム状態取得
SetSecretNumberUseCase 秘密数字登録
StartGameUseCase IN_PROGRESS へ遷移・先手ターン開始
SubmitGuessUseCase 予想・判定・ターン進行
HandleTimeoutUseCase タイムアウト時の自動予想
FinishGameUseCase 終了・MatchHistory・WinCount 加算
CancelGameBySecretTimeoutUseCase 秘密数字期限切れでゲーム終了
SyncGameStateUseCase 再接続同期
RecoverActiveGamesUseCase サーバー起動時の進行中ゲーム復元
プロフィール・履歴
UseCase 概要
GetProfileUseCase プロフィール
GetMatchHistoryUseCase 勝敗履歴
GetLoginHistoryUseCase ログイン履歴
GetWebSocketHistoryUseCase WS 接続履歴
ランキング
UseCase 概要

GetRankingUseCase 上位 3 名
RebuildRankingUseCase 再集計
管理者
UseCase 概要
GetAdminUsersUseCase ユーザー一覧
SearchAdminUsersUseCase 検索
DeleteUserUseCase 論理削除
SearchActivityLogsUseCase ログ検索
DownloadActivityLogsUseCase CSV 出力
GetBackupStatusUseCase バックアップ状況

5.2 処理フロー

RegisterUserUseCase

目的

新規ユーザーを登録する

password を bcrypt でハッシュ化して DB 保存

登録情報を返す

行わないこと: JWT 発行 / refresh_token / login_logs / Set-Cookie

入力

項目 型 必須 条件

username string 必須 3 ~ 50 文字、1+\$

項目 型 必須 条件

email string 必須 RFC5322、255 文字以下

password string 必須 8 文字以上

処理手順

1. username / email / password を検証 → validation_error

2. users を username または email で検索

3. 重複 → duplicate_user

4. password を bcrypt cost=12 でハッシュ化5. BEGIN

1. users INSERT (role=user, win_count=0, last_activity_at=now)

7. COMMIT8. Response 201 (§6.3.1)

Response 201

```
{  
  "data": {  
    "id": "uuid",  
    "username": "alice",  
    "role": "user",  
    "winCount": 0  
  }  
}
```

エラー

code 条件

validation_error 入力不正

duplicate_user username / email 重複

rate_limit_exceeded IP 制限

internal_error DB 失敗

LoginUseCase

目的

登録済みユーザーを認証

JWT 発行

refresh_token 発行 → refresh_tokens INSERT → Set-Cookie

login_logs 保存

last_activity_at 更新

削除済みユーザー拒否

入力

項目 型 必須 条件

email string 必須 メール形式、50 文字以下

項目 型 必須 条件

password string 必須 8 文字以上

処理手順

1. email / password 検証 → validation_error
2. users を email + deleted_at IS NULL で検索
3. 不存在 → unauthorized
4. bcrypt 照合 → 不一致 unauthorized
5. jti = UUID v4
6. JWT 発行 (sub, role, jti, iat, exp)

7. refresh_token 生成

crypto/rand 64B → hex

token_hash = SHA256 (平文)

family_id = UUID v4

expires_at = now + REFRESH_TOKEN_EXPIRY_DAYS

8. BEGIN 9. login_logs INSERT (action=login)

10. users.last_activity_at = now

11. refresh_tokens INSERT (status=active)

12. COMMIT 13. Body: { data: { accessToken } }

14. Set-Cookie: refresh_token

refresh_tokens INSERT

カラム 値

token_hash SHA256 (平文トークン)

family_id 新規 UUID v4

status active

expires_at now + REFRESH_TOKEN_EXPIRY_DAYS

Cookie

項目 内容

名前 refresh_token

HttpOnly / Secure / SameSite=true / true / Strict

Path /api/auth/refresh

Max-Age REFRESH_TOKEN_EXPIRY_DAYS 86400

Response 200

Body

```
{ "data": { "accessToken": "jwt" } }
```

Headers

Set-Cookie: refresh_token=; HttpOnly; Secure; SameSite=Strict;

Path=/api/auth/refresh; Max-Age=604800

エラー

code HTTP 条件

validation_error 400 入力不正

unauthorized 401 不存在・不一致・削除済み

rate_limit_exceeded 429 IP 制限

internal_error 500 DB 失敗

LogoutUseCase

目的

ログイン中の JWT を無効化する

リフレッシュトークンを失効させる

WebSocket 接続情報を削除する

ログアウト履歴を保存する

ログアウト後に同じ JWT を再利用できないようにする

入力

項目 型 必須 説明

userID uuid.UUID 必須 JWT から取得したユーザー ID

jti string 必須 JWT に含まれるトークン識別子

exp time.Time 必須 JWT の有効期限

処理手順

JWT を検証する

JWT が無効な場合は unauthorized を返す

JWT から userID を取得する

JWT から jti を取得する

JWT から exp を取得する
jti が空の場合は unauthorized を返す
現在時刻と exp から JWT の残り有効期限を計算する
JWT の有効期限がすでに切れている場合は unauthorized を返す
Redis に失効済み JWT として登録する
キーは jwt:revoked:{jti}
値は 1
TTL は JWT の残り有効期限
Cookie から refresh_token を取得する
refresh_tokens テーブルで該当ユーザーの status = active なレコードをすべて
revoked に更新する
revoked_at = 現在時刻
Cookie が存在しない・DB に対象レコードがない場合はスキップして続行する（ログ
アウトは常に成功させる）
Redis から WebSocket 接続情報を削除する
キーは ws:user:{userID}

DB トランザクションを開始する
login_logs にログアウト履歴を INSERT する
users.last_activity_at を現在時刻に更新する
DB 更新に失敗した場合はロールバックする
DB 更新に成功した場合はコミットする
HTTP 204 を返す
Set-Cookie でリフレッシュトークンを削除する
Redis 更新内容
JWT 失効
項目 内容
キー jwt:revoked:{jti}
値 1
TTL JWT の残り有効期限
用途 ログアウト済み JWT の再利用防止
WebSocket 接続情報
項目 内容
キー ws:user:{userID}
処理 削除
用途 ログアウト後の WebSocket 接続情報を無効化

DB 登録内容

login_logs

カラム 値

id UUID v4

user_id ログアウトユーザー ID

action logout

created_at 現在時刻

users

カラム 値

last_activity_at 現在時刻

updated_at 現在時刻

refresh_tokens

カラム 値

status revoked

revoked_at 現在時刻

updated_at 現在時刻

Cookie

項目 内容

名前 refresh_token

値 空文字

項目 内容

Max-Age 0 (即時削除)

Path /api/auth/refresh

Response

204 No Content

RefreshTokenUseCase

目的

リフレッシュトークンを検証し、新しいアクセストークンを発行する

トークンローテーションにより旧トークンを即時失効させ、新しいリフレッシュトークンを発行する

失効済みトークンの再使用を検出した場合、同一ファミリーの全トークンを失効させる (盗用対策)

削除済みユーザーのトークン更新を拒否する

基本方針

リフレッシュトークンは Cookie から取得する

リクエストボディは使用しない

トークン検証は DB の refresh_tokens テーブルを参照する

失効済みトークンの再使用を検出した場合は family_id が同じ全トークンを失効させる

DB 更新はトランザクション内で行う

新しいリフレッシュトークンは Set-Cookie で返す

入力

項目 型 必須 説明

refreshToken string 必須 Cookie refresh_token から取得したトークン文字列

前提条件

Cookie に refresh_token が存在する

refresh_tokens テーブルに対応するレコードが存在する

トークンの status が active

トークンの expires_at が現在時刻より未来

対象ユーザーが削除済みでない

処理手順

Cookie から refresh_token を取得する

存在しない場合は unauthorized を返す

取得したトークン文字列を SHA256 でハッシュ化する

refresh_tokens テーブルから token_hash で検索する

レコードが存在しない場合は unauthorized を返す

トークンの status が revoked の場合

同じ family_id を持つ status = active なレコードをすべて revoked に更新する

(revoked_at = 現在時刻)

Redis から ws:user:{userID} を削除して WebSocket 接続を切断する

unauthorized を返す

トークンの expires_at が現在時刻以前の場合

status = revoked、revoked_at = 現在時刻 に更新する

unauthorized を返す

user_id でユーザーを取得する

ユーザーが存在しない場合は unauthorized を返す

deleted_at が NULL でない場合はトークンを revoked に更新して unauthorized を返す

新しいアクセストークン (JWT) を発行する

exp は JWT_EXPIRY_MINUTES 分後

jti は新規 UUID v4

新しいリフレッシュトークンを生成する
crypto/rand で 64 バイトのランダム値を生成し hex エンコードする
SHA-256 でハッシュ化して token_hash を生成する
family_id は旧トークンの値を引き継ぐ
expires_at = now + REFRESH_TOKEN_EXPIRY_DAYS 日
DB トランザクションを開始する
旧リフレッシュトークンを失効させる
status = revoked
revoked_at = 現在時刻
updated_at = 現在時刻
refresh_tokens に新しいリフレッシュトークンを INSERT する
users.last_activity_at を現在時刻に更新する
DB 更新に失敗した場合はロールバックして internal_error を返す
DB 更新に成功した場合はコミットする
accessToken を返す
新しいリフレッシュトークンを Set-Cookie でセットする
DB 更新内容
refresh_tokens (旧トークン)
カラム 値
status revoked
revoked_at 現在時刻
updated_at 現在時刻
refresh_tokens (新トークン)
カラム 値
id UUID v4
user_id ユーザー ID
token_hash SHA256 (新しいトークン文字列)
family_id 旧トークンの family_id を引き継ぐ
status active
expires_at now + REFRESH_TOKEN_EXPIRY_DAYS 日

カラム 値
created_at 現在時刻
updated_at 現在時刻
users
カラム 値

last_activity_at 現在時刻

updated_at 現在時刻

Cookie

項目 内容

名前 refresh_token

値 新しいトークン文字列（ハッシュ前の平文）

HttpOnly true

Secure true

SameSite Strict

Path /api/auth/refresh

Max-Age REFRESH_TOKEN_EXPIRY_DAYS × 86400 秒

Response

json

```
{ "data": { "accessToken": "jwt" }}
```

Errors: `unauthorized`, `internal_error`

AutoLogoutUseCase

目的

一定時間操作していないユーザーを自動ログアウトする

放置されたログイン状態を無効化する

WebSocket 接続中のユーザーも強制的に切断する

自動ログアウト履歴を保存する

実行タイミング

AutoLogoutWorker から定期実行

実行間隔は AUTO_LOGOUT_POLL_SECONDS

初期値は 60 秒

Worker 間隔は環境変数から読み込む

コード内に 60 秒 を直接書かない

対象条件

last_activity_at < now - SESSION_TIMEOUT_MINUTES

入力

項目 型 必須 説明

now time.Time 必須 判定基準時刻

sessionTimeouttime.Duration 必須 無操作タイムアウト時間

処理手順

現在時刻を取得する

SESSION_TIMEOUT_MINUTES を読み込む
自動ログアウト対象時刻を計算する
対象時刻 = now - SESSION_TIMEOUT_MINUTES
users テーブルから対象ユーザーを取得する
deleted_at IS NULL
last_activity_at < 対象時刻
対象ユーザーが存在しない場合は正常終了する
対象ユーザーごとに処理する
Redis に強制ログアウト基準時刻を保存する
キーは user:{userID}:force_logout_before
値は現在時刻の Unix 秒
TTL は 30 日
Redis から WebSocket 接続情報を取得する
キーは ws:user:{userID}
WebSocket 接続が存在する場合は、対象ユーザーへ ERROR イベントを送信する
ERROR イベント送信後、WebSocket 接続を切断する
Redis から ws:user:{userID} を削除する
DB トランザクションを開始する
login_logs に自動ログアウト履歴を INSERT する
必要に応じて users.updated_at を更新する
DB 更新に失敗した場合はロールバックする
DB 更新に成功した場合はコミットする
全ユーザーの処理完了後に正常終了する
対象ユーザー取得 SQL 例
SELECT idFROM usersWHERE deleted_at IS NULL AND last_activity_at <
:timeoutAt;
AuthenticateWebSocketUseCase
目的

WebSocket 接続時にユーザー認証を行う
認証済みユーザーだけ WebSocket イベントを利用可能にする
同一ユーザーの多重接続を防ぐ
WebSocket 接続履歴を保存する
認証成功時に AUTH_OK を返す
実行タイミング
クライアントが /ws に接続した後

クライアントが AUTH イベントを送信した時
WebSocket 接続後 5 秒以内に実行する
5 秒以内に AUTH が送信されない場合は接続を切断する

Client → Server

```
{ "type": "AUTH", "token": "jwt" }
```

StartMatchingUseCase

目的

ユーザーを対戦相手待ちの状態にする

待機中ユーザーが 2 人以上いる場合は、その場で対戦相手を決定する

対戦相手が見つかった場合はゲームを作成する

ゲーム作成に失敗した場合は、待機登録も取り消す

サーバー側で自動再試行は行わない

基本方針

マッチング開始処理の中で、対戦相手の検索まで行う

対戦相手が見つかった場合は、同じ処理内でゲームを作成する

常駐 Worker で後からマッチングする方式は採用しない

処理途中で失敗した場合は、DB 状態を処理前に戻す

再試行はクライアント側から再度リクエストする

入力

項目 型 必須 説明

userID uuid.UUID 必須 マッチング開始ユーザー ID

前提条件

ユーザーがログイン済みであること

ユーザーが削除済みでないこと

ユーザーが master 権限でないこと

ユーザーが対戦中でないこと

ユーザーが秘密数字登録待ちのゲームに参加中でないこと

ユーザーがすでにマッチング待機中でないこと

処理手順

userID でユーザーを取得する

ユーザーが存在しない場合は unauthorized を返す

deleted_at が NULL でない場合は unauthorized を返す

role = master の場合は forbidden を返す

対戦中ゲームを確認する

WAITING_SECRET のゲームに参加中なら user_in_active_game

IN_PROGRESS のゲームに参加中なら user_in_active_game
matching_queue を確認する
すでに待機中なら already_in_matching を返す
DB トランザクションを開始する
matching_queue に待機データを INSERT する
user_id = userID
status = waiting
created_at = now
MatchPlayersUseCase を同じトランザクション内で呼び出す
待機中ユーザーが 2 人未満の場合
ゲームは作成しない
ユーザーは待機状態のままにする
待機中ユーザーが 2 人以上の場合
登録順に先頭 2 人を取得する
先に待機したユーザーを player1 にする
後から待機したユーザーを player2 にする
games に新規ゲームを作成する
status = WAITING_SECRET にする
current_turn = 1 にする
player1 と player2 を matching_queue から削除する
MatchPlayersUseCase 内で DB エラーが発生した場合
トランザクション全体をロールバックする
手順 9 の matching_queue INSERT も取り消す
internal_error を返す
すべて成功した場合は COMMIT する
マッチング成立時のみ、COMMIT 後に両プレイヤーへ MATCHED を送信する
レスポンスを返す
対戦中チェック SQL

```
SELECT idFROM gamesWHERE status IN ('WAITING_SECRET', 'IN_PROGRESS')
AND (player1_id = :userID OR player2_id = :userID);
```

MatchPlayersUseCase

目的

マッチング待機中のユーザーを 2 人ずつ組み合わせる

先に待機したユーザーを player1 にする

後から待機したユーザーを player2 にする

対戦用の Game を作成する

マッチング成立後、両ユーザーを待機一覧から削除する

基本方針

待機中ユーザーは登録順に処理する

created_at が古いユーザーを優先する

先に待機したユーザーを player1 とする

player1 を先手とする

player2 を後手とする

待機中ユーザーが 2 人未満の場合は何もせず正常終了する

DB 更新はトランザクション内で行う

WebSocket 通知は DB コミット後に行う

入力

項目 型 必須 説明

ctx context.Context 必須 リクエストコンテキスト

tx Transaction 必須 呼び出し元から渡された DB トランザクション

出力

項目 型 説明

GameDTO GameDTO / null マッチング成立時のみ返す

error error 処理失敗時に返す

前提条件

StartMatchingUseCase から同一トランザクション内で呼び出す

matching_queue に待機ユーザーが登録されている

削除済みユーザーは待機登録されていない前提

master ユーザーは待機登録されていない前提

対戦中ユーザーは待機登録されていない前提

処理手順

matching_queue から待機中ユーザーを取得する

条件は status = waiting

並び順は created_at ASC

取得件数は 2 件

SELECT FOR UPDATE で取得行をロックする

待機中ユーザーが 2 件未満の場合

ゲームは作成しない

待機一覧は変更しない

GameDTO = null で正常終了する

2 件取得できた場合
1 件目を player1 とする
2 件目を player2 とする
player1 と player2 が同一ユーザーでないことを確認する
同一の場合は internal_error
念のため、player1 と player2 が削除済みでないことを確認する
削除済みなら該当ユーザーの待機データを削除する
この場合はゲームを作成せず正常終了する
念のため、player1 と player2 が master 権限でないことを確認する
master なら該当ユーザーの待機データを削除する
この場合はゲームを作成せず正常終了する
player1 と player2 が対戦中でないことを再確認する
WAITING_SECRET または IN_PROGRESS のゲームに参加中なら該当ユーザーを待機一覧から削除する
この場合はゲームを作成せず正常終了する
games テーブルに新規ゲームを INSERT する
status = WAITING_SECRET
player1_id = player1
player2_id = player2
current_turn = 1
current_turn_player_id = NULL
winner_id = NULL
started_at = NULL
finished_at = NULL
matching_queue から player1 と player2 の待機データを削除する
GameDTO を生成する
呼び出し元に GameDTO を返す
待機ユーザー取得 SQL
SELECT id, user_id, created_at FROM matching_queue

WHERE status = 'waiting' ORDER BY created_at ASC LIMIT 2 FOR UPDATE;
SetSecretNumberUseCase

目的

プレイヤーが自分の秘密数字を登録する
秘密数字登録フェーズ中のみ登録を許可する
同じプレイヤーによる二重登録を防ぐ

2 人の秘密数字がそろった場合にゲーム開始処理へ進める
相手の秘密数字をレスポンス・ログ・通知に含めない

基本方針

秘密数字は WAITING_SECRET 状態のゲームでのみ登録可能
登録できるのはゲーム参加者のみ

1 プレイヤーにつき秘密数字登録は 1 回のみ

秘密数字は 4 桁固定

各桁は 0 ～ 9

同じ数字は使用不可

連打・二重送信は Redis ロックで防止

DB 更新はトランザクション内で行う

WebSocket 通知は DB コミット後に送信する

両者登録完了後に StartGameUseCase を実行する

入力

項目 型 必須 説明

gameID uuid.UUID 必須 対象ゲーム ID

userID uuid.UUID 必須 秘密数字を登録するユーザー ID

secretNumber string / 4]int 必須 登録する 4 桁数字

前提条件

ユーザーがログイン済み

ユーザーが削除済みでない

対象ゲームが存在する

対象ゲームの status が WAITING_SECRET

ユーザーが player1 または player2

自分の秘密数字が未登録

入力された秘密数字が正しい形式

処理手順

Redis キー game:{gameID}:player:{userID}:secret_lock を Lua で acquireTTL
GAME_LOCK_SECONDS

ロック失敗 → rate_limit_exceeded

NewSecretNumber で形式検証

SecretHashService.Hash(secretNumber, gameID, playerSlot)

平文 secretNumber は以降参照しない

BEGIN → games FOR UPDATE

検証 : 参加者 / WAITING_SECRET / 未登録

player1_secret または player2_secret にハッシュ文字列を保存

Game.SetSecretHash(playerID, hash)

COMMIT

片方のみ → WS なし

両者完了 → StartGameUseCase

Redis

項目 内容

キー game:{gameID}:player:{userID}:secret_lock

値 1

TTL GAME_LOCK_SECONDS

目的 秘密数字登録の二重送信防止

Game 取得 SQL

SELECT *FROM gamesWHERE id = :gameIDFOR UPDATE;

StartGameUseCase

目的

両プレイヤーの秘密数字登録完了後にゲームを開始する

ゲーム状態を WAITING_SECRET から IN_PROGRESS に変更する

先手プレイヤーを player1 に設定する

1 ターン目の制限時間を Redis に保存する

両プレイヤーへゲーム開始とターン開始を通知する

ゲーム開始ログを保存する

基本方針

ゲーム開始は両プレイヤーの秘密数字が登録済みの場合のみ実行する

player1 を必ず先手とする

currentTurn は 1 から開始する

currentTurnPlayerID は player1_id にする

DB 更新が成功してから Redis 更新と WebSocket 通知を行う

DB 保存に失敗した場合は、Redis 更新と WebSocket 通知を行わない

WebSocket 通知は DB コミット後に送信する

入力

項目 型 必須 説明

gameID uuid.UUID 必須 開始対象のゲーム ID

前提条件

対象ゲームが存在する

ゲーム状態が WAITING_SECRET

player1_secret が登録済み
player2_secret が登録済み
started_at が未設定
finished_at が未設定
winner_id が未設定
処理手順
DB トランザクションを開始する
games を gameId で取得する
処理中は他のリクエストが同じゲーム情報を変更できないようにする
ゲームが存在しない場合は処理を中止し、変更を保存せずに not_found を返す
status が WAITING_SECRET でない場合は処理を中止し、変更を保存せずに validation_error を返す
player1_secret が未登録の場合は処理を中止し、変更を保存せずに validation_error を返す
player2_secret が未登録の場合は処理を中止し、変更を保存せずに validation_error を返す
started_at が既に設定されている場合は処理を中止し、変更を保存せずに validation_error を返す
ゲーム開始情報を設定する
status = IN_PROGRESS
current_turn = 1
current_turn_player_id = player1_id
started_at = 現在時刻
updated_at = 現在時刻
games を UPDATE する
opponentSecretHash を DB から取得 player1 → player2_secret / player2 → player1_secret
未登録 → validation_error
digitResults ← SecretHashService.Verify(...)
hitCount ← digitResults の 1 の個数
isWin ← Domain.IsWin(digitResults)
guesses INSERT

isWin → FinishGameService / 未勝利 → 次ターン UPDATE
COMMIT → WS 通知
Game 取得 SQL

```
SELECT *FROM gamesWHERE id = :gameIDFOR UPDATE;
```

SubmitGuessUseCase

目的

プレイヤーの予想を受け付ける

予想数字を検証する

相手の秘密数字と比較して各桁の判定結果を作る

勝利条件を満たすか判定する

勝利していない場合は次プレイヤーへターンを移す

勝利した場合はゲーム終了処理（match_histories 作成・win_count 加算・status 更新）を同一トランザクション内で完結させる

予想結果を両プレイヤーへリアルタイム送信する

勝利終了処理 games FINISHED / match_histories / win_count は

FinishGameService 経由のみ

原子性 guesses INSERT と勝利終了（または次ターン UPDATE）は 同一 TX

基本方針

予想できるのはゲーム参加者のみ

予想できるのは自分のターンのみ

予想できるのは IN_PROGRESS のゲームのみ

予想数字は 4 桁固定

各桁は 0 ～ 9

同じ数字は使用不可

連打・二重送信は Redis で防止する

桁判定と勝敗判定は Domain で行う

DB 更新はトランザクション内で行う

勝利時の終了処理（match_histories・win_count・status = FINISHED）は同一トランザクション内で完結させる

FinishGameUseCase は独立したトランザクションとして呼び出さない

WebSocket 通知は DB コミット後に送信する

入力

項目 型 必須 説明

gameID uuid.UUID 必須 対象ゲーム ID

userID uuid.UUID 必須 予想を送信したユーザー ID

guessNumberstring / 4]int 必須 予想した 4 桁数字

項目 型 必須 説明

isAuto bool 任意 タイムアウト自動予想の場合は true

前提条件

ユーザーがログイン済み

ユーザーが削除済みでない

対象ゲームが存在する

対象ゲームの状態が IN_PROGRESS

ユーザーがゲーム参加者

ユーザーが現在ターンのプレイヤー

相手の秘密数字が登録済み

予想数字が正しい形式

処理手順

予想送信の二重実行を防ぐため、Redis キー game:{gameID}:player:

{userID}:guess_lock を Lua で acquireTTL GAME_LOCK_SECONDS既に処理中の場合は rate_limit_exceeded を返す

guessNumber を検証する

4 桁でない場合は invalid_digit_length

数字以外を含む場合は invalid_digit

同じ数字を含む場合は duplicate_digit

NewGuessNumber(guessNumber) を呼び出す

GuessNumber の生成に失敗した場合は対応するバリデーションエラーを返す

DB トランザクションを開始する

games を gameID で取得する

処理中は他のリクエストが同じゲーム情報を変更できないようにする

ゲームが存在しない場合は処理を中止し、変更を保存せずに not_found を返す

status が IN_PROGRESS でない場合は処理を中止し、変更を保存せずに

game_not_in_progress を返す

ユーザーがこのゲームの参加者でない場合は、変更を保存せずに処理を終了し、

forbidden を返す

ユーザーが現在ターンのプレイヤーでない場合は、変更を保存せずに処理を終了し、

not_your_turn を返す

予想対象となる相手の秘密数字を取得する

userID == player1_id の場合は player2_secret

userID == player2_id の場合は player1_secret

相手の秘密数字が未登録の場合は、変更を保存せずに処理を終了し、validation_error を返す

Domain の桁判定関数を呼び出す

秘密数字と予想数字を同じ位置ごとに比較する

一致なら 1
不一致なら 0
hitCount を計算する
digitResults の 1 の数を数える
Domain の勝敗判定を行う

hitCount == 4 の場合は勝利
hitCount < 4 の場合はゲーム継続
Guess を生成する
guesses テーブルに INSERT する
guesses テーブルに INSERT する
Domain.IsWin の結果が true の場合、同一トランザクション内で
FinishGameService(ctx, tx, game, userID を呼び出す
FinishGameService が error を返した場合、ROLLBACK し internal_error を返す（
guesses INSERT も取り消される）
Domain.IsWin の結果が false の場合、未勝利処理を行う（current_turn 1、
current_turn_player_id を相手に変更）
games テーブルを UPDATE する（未勝利時のみ。勝利時は FinishGameService 内）
DB 更新に失敗した場合、ROLLBACK し internal_error を返す
DB トランザクションを COMMIT する
activity_logs に guess を保存する
勝利時は両プレイヤーへ GAME_OVER を送信する
未勝利時は次ターン情報を Redis に保存する
未勝利時は両プレイヤーへ TURN_CHANGED を送信する
GuessResultDTO を返す
Redis
項目 内容
キー game:{gameID}:player:{userID}:guess_lock
値 1
TTL GAME_LOCK_SECONDS
目的 予想送信の二重実行防止（手動 GUESS と HandleTimeoutUseCase で共用）
Game 取得 SQL
sql

```
SELECT *FROM gamesWHERE id = :gameIDFOR UPDATE;
```


DB 更新内容（勝利時）
games

カラム 値
status FINISHED
winner_id winnerID
finished_at 現在時刻
current_turn_player_id NULL
updated_at 現在時刻

match_histories

カラム 値
id UUID v4
game_id gameId
winner_id winnerID
loser_id loserID
winner_username 勝者のユーザー名
loser_username 敗者のユーザー名
finished_at 現在時刻
users (勝者)
カラム 値
win_count win_count + 1
updated_at 現在時刻
HandleTimeoutUseCase

目的

制限時間内に予想しなかったプレイヤーの代わりに自動予想を実行する
自動予想を通常の予想と同じルールで判定する
手動予想とタイムアウト処理の二重実行を防ぐ
タイムアウト後に次ターンへ進める
自動予想で勝利した場合はゲーム終了処理へ進める

基本方針

ターン期限を過ぎた場合のみ実行する
自動予想は 4 桁固定
各桁は 0 ～ 9
同じ数字は使用不可
自動予想は isAuto = true として保存する
手動予想と同じ Redis キーで二重実行を防ぐ
DB 更新はトランザクション内で行う
Game 状態は DB 取得後に再確認する

WebSocket 通知は DB コミット後に送信する
自動予想でも勝利条件を満たせば勝利扱いにする
自動予想は 0000 ～ 9999 のうち、重複する数字を含まない組み合わせから 1 件を選択する
自動予想生成は RandomNumberService 経由で行う
入力
項目 型 必須 説明
gameID uuid.UUID 必須 タイムアウト対象ゲーム ID
playerID uuid.UUID 必須 タイムアウト対象プレイヤー ID
now time.Time 必須 判定基準時刻

前提条件
対象ゲームが存在する
ゲーム状態が IN_PROGRESS
playerID が現在ターンのプレイヤー
Redis に現在ターン情報が存在する
Redis の expiresAt が現在時刻を過ぎている
同じターンで手動予想がまだ処理されていない
処理手順
手動予想と同一キー game:{gameID}:player:{playerID}guess_lock を Lua で
acquireTTL GAME_LOCK_SECONDS既に処理中の場合は、何もせず正常終了する
Redis から現在ターン情報を取得する
キーは game:{gameID}:turn
Redis にターン情報が存在しない場合
何もせず正常終了する
Redis の playerId が playerID と一致しない場合
何もせず正常終了する
Redis の expiresAt が現在時刻以降の場合
まだ期限切れではないため、何もせず正常終了する
DB トランザクションを開始する
games を gameID で取得する
処理中は他のリクエストが同じゲーム情報を変更できないようにする
ゲームが存在しない場合
処理を中止する
変更を保存せず正常終了する
status が IN_PROGRESS でない場合

処理を中止する
変更を保存せず正常終了する
current_turn_player_id が playerId と一致しない場合
既に別プレイヤーへターンが移っている可能性があるため、変更を保存せず正常終了する
DB 上の current_turn と Redis 上の turn が一致しない場合
古いタイムアウト処理の可能性があるため、変更を保存せず正常終了する
Redis の expiresAt を再確認する
DB ロック取得中に手動予想が完了している可能性を考慮する
期限切れでない場合
変更を保存せず正常終了する
RandomNumberService.GenerateGuessNumber() を呼び出す
重複なし 4 桁の自動予想を生成する
予想対象となる相手の秘密数字を取得する

playerID == player1_id の場合は player2_secret
playerID == player2_id の場合は player1_secret
Domain の桁判定関数を呼び出す
hitCount を計算する
Domain の勝敗判定を行う
hitCount == 4 の場合は勝利
hitCount < 4 の場合はゲーム継続
Guess を生成する
isAuto = true
guesses テーブルに INSERT する
Domain.IsWin が true → 同一 TX 内で FinishGameService(ctx, tx, game, playerId)
23.FinishGameService が error → ROLLBACK、internal_error
false → 未勝利処理 (current_turn 1)。Redis ターンは COMMIT 後
25.DB 更新に失敗した場合
処理を中止する
変更を保存せず internal_error を記録する
26.DB トランザクションを確定する
27.Redis の現在ターン情報を削除する

28. 両プレイヤーへ GUESS_RESULT を送信する

activity_logs に timeout を保存する

30. 勝利時は両プレイヤーへ GAME_OVER を送信する

31. 未勝利時は次ターン情報を Redis に保存する

32. 未勝利時は両プレイヤーへ TURN_CHANGED を送信する

Redis

項目 内容

キー game:{gameID}:player:{playerID}:guess_lock

値 1

TTL GAME_LOCK_SECONDS

目的 手動予想とタイムアウト自動予想の二重実行防止

Redis ターン情報

キー

game:{gameID}:turn

FinishGameUseCase

FinishGameUseCase は外部から独立して呼び出す UseCase ではなく、SubmitGuessUseCase / HandleTimeoutUseCase の同一ランザクション内から呼び出す内部ドメインサービスとして再定義する。
呼び出し元のランザクションを tx として受け取り、自身ではランザクションを開始・終了しない。

目的

予想によって勝者が決まったゲームの終了処理をひとまとめに実行する

ゲーム状態を FINISHED に変更する

勝敗履歴を保存する

勝者の勝利数を加算する

呼び出し元のランザクション内で完結させ、DB コミットの原子性を保証する

基本方針

勝者が確定している場合のみ実行する

ゲーム状態が IN_PROGRESS の場合のみ終了処理を行う

勝者は player1 または player2 のどちらかである必要がある

敗者は勝者ではないもう一方のプレイヤーとする
match_histories は勝敗が確定した場合のみ作成する
勝者の win_count を 1 増やす
自身ではトランザクションを開始・終了しない（呼び出し元の tx を使用する）
WebSocket 通知（GAME_OVER 送信）・activity_logs 保存は呼び出し元が DB コミット後に行う
秘密数字は履歴・ログ・通知に含めない
シグネチャ

go

```
func FinishGameService( ctx context.Context, tx Transaction, game *Game, // 呼び出し元が SELECT FOR UPDATE で取得済みの Game エンティティ winnerID uuid.UUID, ) error
```

入力

項目 型 必須 説明

ctx context.Context 必須 リクエストコンテキスト

tx Transaction 必須 呼び出し元から渡された DB トランザクション

game Game 必須 SELECT FOR UPDATE で取得済みの Game エンティティ

winnerID uuid.UUID 必須 勝者ユーザー ID

処理手順

winnerID が空の場合

error を返す

winnerID が player1_id / player2_id のどちらにも一致しない場合

error を返す

敗者 ID を決定する

winnerID == player1_id の場合は loserID = player2_id

winnerID == player2_id の場合は loserID = player1_id

勝者ユーザーを取得する

敗者ユーザーを取得する

勝者または敗者が存在しない場合

error を返す

game.Finish(winnerID) を呼び出す

games を更新する

status = FINISHED

winner_id = winnerID

finished_at = 現在時刻

current_turn_player_id = NULL

updated_at = 現在時刻
match_histories に勝敗履歴を INSERT する
勝者の win_count を +1 する
users の勝者レコードを更新する
DB エラーが発生した場合は error を返す（ロールバックは呼び出し元が行う）
CancelGameBySecretTimeoutUseCase

目的

秘密数字登録の制限時間を過ぎたゲームを終了する
両プレイヤーの秘密数字がそろわなかったゲームを無効終了にする
勝者なし・敗者なしでゲームを終了する
勝敗履歴を作成しない
勝利数を加算しない
両プレイヤーへゲーム終了を通知する

基本方針

対象は WAITING_SECRET のゲームのみ
IN_PROGRESS のゲームは対象外
FINISHED のゲームは対象外
秘密数字登録期限を過ぎた場合のみ終了する
勝者は設定しない
敗者も設定しない
match_histories は作成しない
win_count は加算しない
DB 更新はトランザクション内で行う
WebSocket 通知は DB コミット後に送信する
入力

項目 型 必須 説明

gameID uuid.UUID 必須 終了対象のゲーム ID

項目 型 必須 説明

now time.Time 必須 判定基準時刻

前提条件

対象ゲームが存在する
ゲーム状態が WAITING_SECRET
created_at + SECRET_SETUP_SECONDS < now
started_at が未設定
finished_at が未設定

winner_id が未設定

処理手順

DB トランザクションを開始する

games を gameId で取得する

処理中は他のリクエストが同じゲーム情報を変更できないようにする

ゲームが存在しない場合

処理を中止する

変更を保存せず正常終了する

status が WAITING_SECRET でない場合

処理を中止する

変更を保存せず正常終了する

started_at が設定済みの場合

すでにゲーム開始済みのため正常終了する

finished_at が設定済みの場合

すでに終了済みのため正常終了する

秘密数字登録期限を計算する

期限 = created_at + SECRET_SETUP_SECONDS

現在時刻が期限以内の場合

まだ期限切れではないため正常終了する

game.Finish(nil) を呼び出す

games を更新する

status = FINISHED

winner_id = NULL

finished_at = 現在時刻

updated_at = 現在時刻

match_histories は作成しない

users.win_count は更新しない

DB 更新に失敗した場合

処理を中止する

変更を保存せず internal_error を記録する

DB トランザクションを確定する

Redis に game:{gameID}:turn が存在する場合は削除する

両プレイヤーへ GAME_OVER を送信する

activity_logs に game_over を保存する

正常終了する

Game 取得 SQL

```
SELECT *FROM gamesWHERE id = :gameIDFOR UPDATE;
```

SyncGameStateUseCase

目的

クライアントへ現在のゲーム状態を再送信する

WebSocket 再接続後に画面状態を復元する

ページ再読み込み後に最新状態を取得する

サーバー側の正しいゲーム状態をクライアントへ返す

クライアント側の古い表示状態を修正する

基本方針

ゲーム状態の保存は PostgreSQL とする

Redis はターン残り時間の補助情報として使用する

WebSocket は状態通知にのみ使用する

参加者以外にはゲーム状態を返さない

自分の予想履歴のみ返す

相手の秘密数字は返さない

相手の予想内容は返さない

相手の予想回数のみ返す

秘密数字はレスポンス・ログ・WebSocket 通知に含めない

入力

項目 型 必須 説明

gameID uuid.UUID 必須 同期対象のゲーム ID

userID uuid.UUID 必須 同期要求ユーザー ID

前提条件

ユーザーがログイン済み

ユーザーが削除済みでない

対象ゲームが存在する

ユーザーが対象ゲームの参加者である

処理手順

games を gameID で取得する

ゲームが存在しない場合は not_found を返す

userID が player1_id / player2_id のどちらにも一致しない場合は forbidden を返す

自分のプレイヤー種別を判定する

userID == player1_id の場合は自分を player1 とする

userID == player2_id の場合は自分を player2 とする

guesses から自分の予想履歴のみ取得する
条件は game_id = gameId
条件は player_id = userID
並び順は turn ASC
guesses から相手の予想件数を取得する
条件は game_id = gameId
条件は player_id != userID
Redis から現在ターン情報を取得する
キーは game:{gameID}:turn
Redis にターン情報がある場合
expiresAt - 現在時刻 で残り秒数を計算する
残り秒数が 0 未満の場合は 0 にする
Redis にターン情報がない場合
remainingSeconds = 0 にする
GameStateDTO を作成する
WebSocket 経由の場合は GAME_STATE_SYNC を送信する
HTTP 経由の場合は GameStateDTO をレスポンスとして返す
Game 取得 SQL
SELECT *FROM gamesWHERE id = :gameID;
RecoverActiveGamesUseCase
目的
サーバー再起動後に進行中ゲームのターン情報を復元する
Redis 上のターン情報が消えていても、DB 上のゲーム状態から復元する
期限切れの秘密数字登録待ちゲームを終了する
サーバー再起動によるゲーム停止・画面不整合を防ぐ
実行タイミング
サーバー起動時に 1 回だけ実行する

常駐 Worker として繰り返し実行しない
アプリケーション起動処理の中で実行する
DB 接続と Redis 接続が完了した後に実行する
WebSocketHub が初期化された後に実行する
基本方針
ゲーム状態の保存は PostgreSQL とする
Redis はターン期限の補助情報として扱う
Redis に情報が残っている場合は可能な限り維持する

Redis に情報がない場合は DB の `current_turn_player_id` から再設定する

IN_PROGRESS のゲームのみターン情報を復元する

WAITING_SECRET で秘密数字登録期限切れのゲームは終了する

FINISHED のゲームは対象外にする

入力

項目 型 必須 説明

`now` `time.Time` 必須 起動時の現在時刻

対象ゲーム

進行中ゲーム

`games.status = IN_PROGRESS`

秘密数字登録待ち期限切れゲーム

`games.status = WAITING_SECRET`

`created_at + SECRET_SETUP_SECONDS < now`

処理手順

`games` から進行中ゲームを全件取得する

条件は `status = IN_PROGRESS`

取得したゲームが 0 件の場合

進行中ゲームの復元処理は終了する

各進行中ゲームについて処理する

Redis からターン情報を取得する

キーは `game:{gameID}:turn`

Redis にターン情報が存在する場合

`expiresAt` を確認する

`expiresAt > now` の場合

まだ有効なターン情報としてそのまま維持する

Redis は更新しない

WebSocket 通知は送らない

Redis にターン情報が存在しない場合

DB の `current_turn_player_id` を使用してターン情報を再作成する

Redis の `expiresAt <= now` の場合

期限切れのターン情報として扱う

DB の `current_turn_player_id` を使用してターン情報を再作成する

ターン情報を再作成する場合

`turn = games.current_turn`

`playerId = games.current_turn_player_id`

startedAt = now
expiresAt = now + TURN_DURATION_SECONDS
Redis に game:{gameID}:turn を保存する
対象プレイヤーへ TURN_CHANGED を送信する
activity_logs に復元ログを保存する
次のゲームを処理する
進行中ゲームの復元完了後、秘密数字登録待ちゲームを確認する
WAITING_SECRET かつ秘密数字登録期限切れのゲームを取得する
対象ゲームごとに CancelGameBySecretTimeoutUseCase を呼び出す
すべての処理が完了したら正常終了する

IN_PROGRESS 取得 SQL

```
SELECT *FROM gamesWHERE status = 'IN_PROGRESS';
```

DeleteUserUseCase

目的

管理者が一般ユーザーを削除する
削除対象ユーザーをログイン不可にする
削除対象ユーザーをマッチング対象から外す
対戦中ユーザーの削除を防ぐ
管理者自身の削除を防ぐ
master アカウントの削除を防ぐ
ユーザー削除操作をログに保存する

基本方針

物理削除は行わない
users.deleted_at に削除日時を保存する
users.deleted_by に削除した管理者 ID を保存する
削除済みユーザーはログイン不可
削除済みユーザーはマッチング不可
削除済みユーザーはランキング対象外

対戦中ユーザーは削除不可
master アカウントは削除不可
管理者は自分自身を削除不可

入力

項目 型 必須 説明

adminID uuid.UUID 必須 削除操作を行う管理者 ID

targetID uuid.UUID 必須 削除対象ユーザー ID

前提条件

操作ユーザーがログイン済み

操作ユーザーの role = master

削除対象ユーザーが存在する

削除対象ユーザーが master ではない

削除対象ユーザーが管理者自身ではない

削除対象ユーザーが未削除

削除対象ユーザーが対戦中ではない

処理手順

ユーザー削除の二重実行を防ぐキー : admin:{adminIDuser_delete_lockTTL

ADMIN_LOCK_SECONDS既に処理中の場合は rate_limit_exceeded を返す

adminID で管理者ユーザーを取得する

管理者ユーザーが存在しない場合は unauthorized を返す

管理者ユーザーが削除済みの場合は unauthorized を返す

管理者ユーザーの role が master でない場合は forbidden を返す

targetID で削除対象ユーザーを取得する

削除対象ユーザーが存在しない場合は not_found を返す

targetID == adminID の場合は cannot_delete_self を返す

削除対象ユーザーの role が master の場合は cannot_delete_master を返す

削除対象ユーザーの deleted_at が NULL でない場合は user_already_deleted を返す

DB トランザクションを開始する

削除対象ユーザーが参加中のゲームを確認する

WAITING_SECRET

IN_PROGRESS

対象ユーザーが進行中ゲームに参加している場合

変更を保存せずに処理を終了する

user_in_active_game を返す

削除対象ユーザーを削除する

deleted_at = 現在時刻

deleted_by = adminID

updated_at = 現在時刻

matching_queue に削除対象ユーザーが存在する場合は削除する

activity_logs に user_delete を保存する

DB 更新に失敗した場合

処理を中止する

変更を保存せず internal_error を返す

DB トランザクションを確定する

削除対象ユーザーの WebSocket 接続情報が存在する場合は切断する

削除対象ユーザーの Redis 接続情報を削除する

HTTP 204 を返す

Redis

項目 内容

キー admin:{adminIDuser_delete_lock

値 1

TTL ADMIN_LOCK_SECONDS

目的 ユーザー削除処理の二重実行防止

対戦中チェック SQL

SELECT idFROM gamesWHERE status IN ('WAITING_SECRET', 'IN_PROGRESS')

AND (player1_id = :targetID OR player2_id = :targetID)FOR UPDATE;

SearchActivityLogsUseCase / DownloadActivityLogsUseCase

目的

管理者が操作ログを検索できるようにする

管理者が操作ログを CSV 形式で出力できるようにする

不正操作や障害調査に必要なログを確認できるようにする

CSV 出力時も秘密情報を含めない

対象ユーザー

master 権限のユーザーのみ利用可能

一般ユーザーは利用不可

共通フィルタ

項目 型 必須 説明

logType string 任意 ログ種別

userId uuid.UUID 任意 操作ユーザー ID

項目 型 必須 説明

from time.Time 任意 検索開始日時

to time.Time 任意 検索終了日時

page int 任意 ページ番号

limit int 任意 取得件数

フィルタ条件

logType 指定あり

activity_logs.log_type = logType

userId 指定あり

activity_logs.user_id = userId

from 指定あり

activity_logs.created_at >= from

to 指定あり

activity_logs.created_at <= to

page 未指定

1 を使用する

limit 未指定

20 を使用する

limit が 100 を超える場合

100 に丸める

SearchActivityLogsUseCase

目的

条件に一致する操作ログを一覧表示する

管理画面でログ検索できるようにする

処理手順

操作ユーザーを取得する

操作ユーザーが存在しない場合は unauthorized を返す

操作ユーザーが削除済みの場合は unauthorized を返す

操作ユーザーの role が master でない場合は forbidden を返す

入力されたフィルタを検証する

from が to より後の場合は validation_error を返す

page が 1 未満の場合は validation_error を返す

limit が 1 未満の場合は validation_error を返す

limit が 100 を超える場合は 100 に丸める

activity_logs を条件検索する

検索結果は created_at DESC で並べる

総件数を取得する

ActivityLogDTO に変換する

レスポンスを返す

検索 SQL 例

```
SELECT id, user_id, log_type, detail, created_at FROM activity_logs WHERE
(:logType IS NULL OR log_type = :logType) AND (:userId IS NULL OR user_id =
:userId) AND (:from IS NULL OR created_at >= :from) AND (:to IS NULL OR
```

```
created_at <= :to)ORDER BY created_at DESCLIMIT :limitOFFSET :offset;
```

BackupWorker

目的

通常 DB の重要データをバックアップ DB へ同期する

障害時に復旧できる状態を作る

毎回全件コピーせず、変更されたデータのみ同期する

バックアップ処理の負荷を抑える

バックアップ結果を管理画面で確認できるようにする

実行タイミング

1 日 1 回実行する

BACKUP_CRON

実行時刻は環境変数で変更可能とする

初期値は 03:00 UTC

常時実行ではなく、定期実行 Worker として動かす

基本方針

差分同期方式を採用する

前回同期時刻より後に更新された行のみバックアップ対象にする

前回同期時刻は Redis の backup:status に保存する

初回実行時は前回同期時刻が存在しないため全件を対象にする

バックアップ DB へは主キー単位で UPSERT する

ws_connection_logs はバックアップ対象外にする

バックアップ失敗時は最大 3 回まで再試行する

バックアップ成功時は status=ok を保存する

バックアップ失敗時は status=error を保存する

差分同期とは

前回バックアップ以降に変更されたデータだけを同期する方式

毎回すべてのテーブルをコピーしない

updated_at > lastSyncedAt の行だけ取得する

初回のみ全件を取得する

対象テーブル

テーブル 対象 条件

users 対象 updated_at > lastSyncedAt

games 対象 updated_at > lastSyncedAt

guesses 対象 updated_at > lastSyncedAt

match_histories 対象 updated_at > lastSyncedAt

rankings 対象 updated_at > lastSyncedAt
activity_logs 対象 updated_at > lastSyncedAt
login_logs 対象 updated_at > lastSyncedAt
ws_connection_logs対象外 接続履歴は短期ログ扱い
前提条件
通常 DB に接続できる
バックアップ DB に接続できる
Redis に接続できる
対象テーブルに updated_at が存在する
バックアップ DB 側にも同じテーブル構造が存在する
バックアップ DB 側の updated_at にインデックスが設定されている
通常 DB 側の updated_at にも検索用インデックスを設定する
Redis 管理情報
キー
backup:status
RebuildRankingUseCase
目的
users.win_count から rankings を上書き再集計する
削除済みユーザー・master を除外する
HTTP (POST /api/admin/ranking/rebuild) と RankingRebuildWorker の両方から呼ばれる
処理手順
1. Redis キー admin:{adminId}ranking_rebuild_lock を Lua で acquire
TTL ADMIN_LOCK_SECONDS
ロック取得失敗 → rate_limit_exceeded
2. 操作ユーザーが master であることを確認 → 失敗時 forbidden / unauthorized
3. users から win_count を集計 (deleted_at IS NULL 、 role != master)
4. rankings を TRUNCATE または DELETE 後 INSERT (同一 TX)

5. COMMIT

6. HTTP 204 または Worker 正常終了

Redis
項目 内容

キー admin:{adminID}:ranking_rebuild_lock

値 1

TTL ADMIN_LOCK_SECONDS

目的 ランキング再集計の二重実行防止

LogRetentionUseCase

目的

保持期限を超えた古いログを削除する

DB 容量の増加を防ぐ

ログテーブルの検索性能低下を防ぐ

大量削除による DB 負荷を抑える

削除に失敗してもシステム全体を停止させない

基本方針

ログは無制限に保存しない

保持期限を超えたログのみ削除する

一度に大量削除しない

RETENTION_BATCH_SIZE 件ずつ削除する

各削除バッチの間に短い待機時間を入れる

削除失敗時は警告ログに記録する

削除失敗時もアプリケーションを停止しない

削除できなかったログは次回実行時に再度対象とする

実行タイミング

1 日 1 回実行する

実行時刻は深夜帯を推奨する

実行時刻は環境変数で変更可能とする

常時実行ではなく定期実行 Worker から呼び出す

環境変数

変数名 用途 既定値

ACTIVITY_LOG_RETENTION_DAYS activity_logs 保持日数 90

LOGIN_LOG_RETENTION_DAYS login_logs 保持日数 90

WS_LOG_RETENTION_DAYS ws_connection_logs 保持日数 30

RETENTION_BATCH_SIZE 1 回の削除件数 1000

RETENTION_BATCH_SLEEP_MS バッチ間待機時間 100

削除対象

テーブル 削除条件

activity_logs created_at < now - ACTIVITY_LOG_RETENTION_DAYS


```

login_logs created_at < now - LOGIN_LOG_RETENTION_DAYS
ws_connection_logsconnected_at < now - WS_LOG_RETENTION_DAYS
処理手順
現在時刻を取得する
各保持期間を環境変数から読み込む
RETENTION_BATCH_SIZE を環境変数から読み込む
RETENTION_BATCH_SLEEP_MS を環境変数から読み込む
activity_logs の削除期限を計算する
activityLogsDeleteBefore = now - ACTIVITY_LOG_RETENTION_DAYS
login_logs の削除期限を計算する
loginLogsDeleteBefore = now - LOGIN_LOG_RETENTION_DAYS
ws_connection_logs の削除期限を計算する
wsLogsDeleteBefore = now - WS_LOG_RETENTION_DAYS
activity_logs を RETENTION_BATCH_SIZE 件ずつ削除する
1 回の削除後、削除件数を記録する
削除件数が RETENTION_BATCH_SIZE 件未満なら次テーブルへ進む
削除件数が RETENTION_BATCH_SIZE 件と同じなら、まだ削除対象が残っている可能性
があるため繰り返す
各バッチの間に RETENTION_BATCH_SLEEP_MS だけ待機する
login_logs も同じ方式で削除する
ws_connection_logs も同じ方式で削除する
各テーブルの削除件数を集計する
削除結果をログに記録する
正常終了する
activity_logs 削除 SQL 例
DELETE FROM activity_logsWHERE id IN ( SELECT id FROM activity_logs WHERE
created_at < :deleteBefore ORDER BY created_at ASC LIMIT :batchSize);
DownloadActivityLogsUseCase — 処理手順 先頭追記
1. Redis キー admin:{adminIDlog_download_lock を Lua で acquire
TTL ADMIN_LOCK_SECONDS
ロック取得失敗 → rate_limit_exceeded

```

2. 操作ユーザーが master であることを確認

1. SearchActivityLogsUseCase と同一フィルタで activity_logs を取得

4. CSV 生成（秘密数字・JWT・refresh_token を含めない）

5. text/csv でレスポンス

Redis

項目 内容

キー admin:{adminID}:log_download_lock

値 1

TTL ADMIN_LOCK_SECONDS

目的 ログ CSV ダウンロードの二重実行防止

6. HTTP API 6.1.1 レスポンス形式

成功

```
{ "data": { } }
```

エラー

```
{ "error": { "code": "not_your_turn", "message": "It is not your turn." } }
```

項目 型 説明

data object / array エンドポイントごとのペイロード

error.code string 機械可読なエラー識別子

error.message string 人間可読な説明

6.1.2 認証

項目 内容

方式 JWT Bearer

ヘッダー Cookie `access_token` (`credentials: include`)

JWT Claims sub, role, jti, iat, exp

検証順 署名 → exp → jwt:revoked:{jti} → iat < force_logout_before

条件 code HTTP

exp 超過のみ token_expired 404

署名不正・失効・強制ログアウト unauthorized 401

token_expired 受信時、クライアントは POST /api/auth/refresh でトークン更新後にリトライする。

6.1.3 リフレッシュトークン（Cookie）

項目 内容

名前 refresh_token

取得 POST /api/auth/login 成功時に Set-Cookie

更新 POST /api/auth/refresh 成功時に Set-Cookie

削除 POST /api/auth/logout 成功時に Max-Age=0

HttpOnly true

Secure true

SameSite Strict

Path /api/auth/refresh

項目 内容

Max-Age REFRESH_TOKEN_EXPIRY_DAYS × 86400 秒

リフレッシュトークンは Request Body に含めない。DB には SHA256 ハッシュで保存する。

6.1.4 ページング

パラメータ 既定値 上限

page 1 —

limit 20 100

項目 型 説明

items array データ配列

page int 現在ページ

limit int 1 ページ件数

total int 総件数

6.1.5 レート制限

対象 制限

POST /api/auth/login 同一 IP 10 回 / 分

POST /api/auth/register 同一 IP 5 回 / 分

POST /api/auth/refresh 同一 IP 30 回 / 分

その他認証必須 API 同一ユーザー 120 回 / 分

超過時 : 429 rate_limit_exceeded

6.2 エンドポイント一覧

メソッド パス 認証 UseCase Controller

POST /api/auth/register 不要 RegisterUserUseCase AuthController

POST /api/auth/login 不要 LoginUseCase AuthController

POST /api/auth/refresh Cookie RefreshTokenUseCase AuthController

POST /api/auth/logout JWT LogoutUseCase AuthController

GET /api/me JWT GetMeUseCase MeController

POST /api/matching/start JWT StartMatchingUseCase MatchingController

POST /api/matching/cancel JWT CancelMatchingUseCase MatchingController

GET /api/matching/status JWT GetMatchingStatusUseCase MatchingController

GET /api/games/{id} JWT GetGameStateUseCase GameController

GET /api/me/profile JWT GetProfileUseCase MeController

GET /api/me/match-history JWT GetMatchHistoryUseCase MeController

GET /api/me/login-history JWT GetLoginHistoryUseCase MeController

GET /api/me/ws-history JWT GetWebSocketHistoryUseCaseMeController

GET /api/ranking JWT GetRankingUseCase RankingController

GET /api/admin/users master GetAdminUsersUseCase AdminUserController

GET /api/admin/users/search master SearchAdminUsersUseCase

AdminUserController

DELETE /api/admin/users/{id} master DeleteUserUseCase AdminUserController

POST /api/admin/ranking/rebuild master RebuildRankingUseCase

AdminRankingController

GET /api/admin/logs master SearchActivityLogsUseCase AdminLogController

メソッド パス 認証 UseCase Controller

GET /api/admin/logs/download master

DownloadActivityLogsUseCaseAdminLogController

GET /api/admin/backup/status master GetBackupStatusUseCase

AdminBackupController

GET /health 不要 — SystemController

6.3 認証 API6.3.1 POST /api/auth/register

項目 内容

UseCase RegisterUserUseCase

Controller AuthController

認証 不要

Request Body

項目 型 必須 条件

username string 必須 3 ～ 50 文字、2+\$

email string 必須 RFC5322 準拠、 255 文字以下

password string 必須 8 文字以上

```
{
```

```
  "username": "alice",
```

```
  "email": "a@example.com",
```

```
  "password": "secret12"
```

```
}
```

Response 201

項目 型 説明

id string ユーザー ID (UUID)

username string ユーザー名

role string user 固定

winCount int 初期値 0

```
{
```

```
  "data": {
```

```
    "id": "uuid",
```

```
    "username": "alice",
```

```
    "role": "user",
```

```
    "winCount": 0
```

```
  }
```

```
}
```

code HTTP 条件

validation_error 400 入力形式不正

duplicate_user 409 username または email 重複

rate_limit_exceeded429 IP レート制限超過

6.3.2 POST /api/auth/login

項目 内容

UseCase LoginUseCase

Controller AuthController

認証 不要

Request Body

項目 型 必須 条件

email string 必須 メール形式、50 文字以下

password string 必須 8 文字以上

```
{  
  "email": "a@example.com",  
  "password": "secret12"  
}
```

Response 200

項目 型 説明

accessToken string JWT (JWT_EXPIRY_MINUTES 分有効)

```
{  
  "data": {  
    "accessToken": "jwt"  
  }  
}
```

Response Headers

Set-Cookie: refresh_token=; HttpOnly; Secure; SameSite=Strict;

Path=/api/auth/refresh; Max-Age=604800

code HTTP 条件

validation_error 400 入力形式不正

unauthorized 401 ユーザー不存在・パスワード不一致・削除済み

code HTTP 条件

rate_limit_exceeded429 IP レート制限超過

6.3.3 POST /api/auth/refresh

項目 内容

UseCase RefreshTokenUseCase

Controller AuthController

認証 Cookie refresh_token (Body 不使用)

Request

Body なし。Cookie から refresh_token を読み取る。

Response 200

```
{  
  "data": {  
    "accessToken": "jwt"  
  }  
}
```

Response Headers

旧トークンを失効させ、新 refresh_token を Set-Cookie。

code HTTP 条件

unauthorized 401 Cookie なし・トークン無効・失効・期限切れ・削除済みユーザー・盗用検出

internal_error 500 DB 更新失敗

rate_limit_exceeded429 IP レート制限超過

6.3.4 POST /api/auth/logout

項目 内容

UseCase LogoutUseCase

Controller AuthController

認証 JWT 必須

Request

Body なし。

Response 204

ボディなし。

Response Headers

Set-Cookie: refresh_token=; Path=/api/auth/refresh; Max-Age=0

code HTTP 条件

unauthorized 401 JWT 無効

Cookie 不在・DB に refresh レコードなしでも 204 を返す。

6.4 ユーザー API6.4.1 GET /api/me

項目 内容

UseCase GetMeUseCase

Controller MeController

認証 JWT 必須

Response 200

項目 型 説明

id string ユーザー ID

username string ユーザー名

role string user / master

winCount int 累計勝利数

```
{  
  "data": {  
    "id": "uuid",  
    "username": "alice",  
    "role": "user",  
    "winCount": 3  
  }  
}
```

code HTTP

unauthorized 401

token_expired 404

6.5 マッチング API

マッチングは StartMatchingUseCase 内で同期完結する。常駐 Worker は使用しない。成り立ち時の通知はWebSocket MATCHED。

6.5.1 POST /api/matching/start

項目 内容

UseCase StartMatchingUseCase → MatchPlayersUseCase

Controller MatchingController

認証 JWT 必須 (user のみ)

前提条件

ログイン済み・削除済みでない

master 権限でない

対戦中 (WAITING_SECRET / IN_PROGRESS) でない

待機中でない

Request

Body なし。

Response 200

項目 型 説明

status string waiting 固定

```
{  
  "data": {  
    "status": "waiting"  
  }  
}
```

code HTTP 条件

user_in_active_game409 対戦中

already_in_matching409 既に待機中

forbidden 403 master 権限

internal_error 500 ゲーム作成失敗（待機登録もロールバック）

unauthorized 401 JWT 無効

token_expired 404 JWT 期限切れ

備考

即時ペアリング成功時も HTTP レスポンスは waiting。ゲーム ID は GET /api/matching/status または WS MATCHED で取得する。

6.5.2 POST /api/matching/cancel

項目 内容

UseCase CancelMatchingUseCase

Controller MatchingController

認証 JWT 必須

前提条件

マッチング待機中

Response 200

項目 型 説明

status string cancelled 固定

```
{  
  "data": {  
    "status": "cancelled"  
  }  
}
```

}

}

6.5.3 GET /api/matching/status

項目 内容

UseCase GetMatchingStatusUseCase

Controller MatchingController

認証 JWT 必須

Response 200

項目 型 説明

status string idle / waiting / matched

gameId string | null matched 時のみ UUID

```
{
```

```
  "data": {
```

```
    "status": "waiting",
```

```
    "gameId": null
```

```
  }
```

```
}
```

status 意味

idle 待機・対戦ともに未参加

waiting マッチング待機中

matched ペアリング済み

6.6 ゲーム API

6.6.1 GET /api/games/{id}

項目 内容

UseCase GetGameStateUseCase

Controller GameController

認証 JWT 必須

Path Parameters

項目 型 説明

id UUID ゲーム ID

Response 200 — GameStateDTO

項目 型 説明

gameId string ゲーム ID

項目 型 説明

status string WAITING_SECRET / IN_PROGRESS / FINISHED

currentTurn int 現在ターン（1 始まり）
currentTurnPlayerID string 現在ターンのプレイヤー ID
remainingSeconds int ターン残り秒数（Redis 由来。未設定時 0）
myGuesses GuessSummaryDTO 自分の予想履歴
opponentGuessCount int 相手の予想回数（内容は非公開）
GuessSummaryDTO

項目 型 説明

turn int ターン番号
guessNumber string 予想 4 桁
digitResults int[] 0= 外れ / 1= 当たり
hitCount int 当たり数（0 ～ 4）
isAuto bool タイムアウト自動送信か

```
{  
  "data": {  
    "gameId": "uuid",  
    "status": "IN_PROGRESS",  
    "currentTurn": 3,  
    "currentTurnPlayerID": "uuid",  
    "remainingSeconds": 25,  
    "myGuesses": [  
      {  
        "turn": 1,  
        "guessNumber": "5678",  
        "digitResults": [0, 0, 0, 0],  
        "hitCount": 0,  
        "isAuto": false  
      }  
    ],  
    "opponentGuessCount": 2  
  }  
}
```

}
code HTTP 条件
not_found 404 ゲーム不存在
forbidden 403 参加者でない
相手の秘密数字・相手の予想内容は含めない。

6.7 プロフィール・履歴 API 6.7.1 GET /api/me/profile

項目 内容

UseCase GetProfileUseCase

Controller MeController

認証 JWT 必須

Response 200

項目 型 説明

username string ユーザー名

winCount int 累計勝利数

rank int | null 順位。圏外は null

```
{  
  "data": {  
    "username": "alice",  
    "winCount": 3,  
    "rank": 2  
  }  
}
```

6.7.2 GET /api/me/match-history

項目 内容

UseCase GetMatchHistoryUseCase

Query page, limit

認証 JWT 必須

items[]

項目 型 説明

gameId string ゲーム ID

winnerUsername string 勝者名

loserUsername string 敗者名

項目 型 説明

finishedAt string ISO8601

```
{  
  "data": {  
    "items": [  

```

```
{
  "gameId": "uuid",
  "winnerUsername": "alice",
  "loserUsername": "bob",
  "finishedAt": "2026-06-18T12:00:00Z"
}],
"page": 1,
"limit": 20,
"total": 42
}
}
```

secret_setup_timeout で終了したゲームは含まない。

6.7.3 GET /api/me/login-history

項目 内容

UseCase GetLoginHistoryUseCase

Query page, limit

認証 JWT 必須

items[]

項目 型 説明

action string login / logout

createdAt string ISO8601

```
{
  "data": {
    "items": [
      { "action": "login", "createdAt": "2026-06-18T10:00:00Z" },
      { "action": "logout", "createdAt": "2026-06-18T11:00:00Z" }
    ],
    "page": 1,
    "limit": 20,
    "total": 10
  }
}
```

保持期間 : 90 日。

6.7.4 GET /api/me/ws-history

項目 内容

UseCase GetWebSocketHistoryUseCase

Query page, limit

認証 JWT 必須

items[]

項目 型 説明

connectionId string 接続 ID

connectedAt string ISO8601

disconnectedAt string | null 切断日時

```
{
  "data": {
    "items": [
      {
        "connectionId": "abc123",
        "connectedAt": "2026-06-18T10:00:00Z",
        "disconnectedAt": "2026-06-18T10:30:00Z"
      }
    ],
    "page": 1,
    "limit": 20,
  },
  "total": 5
}
```

保持期間 : 30 日。バックアップ対象外。

6.8 ランキング API6.8.1 GET /api/ranking

項目 内容

UseCase GetRankingUseCase

Controller RankingController

認証 JWT 必須

Response 200 — data[]

項目 型 説明

rank int 順位

```
username string ユーザー名
winCount int 勝利数
{
  "data": [
    { "rank": 1, "username": "alice", "winCount": 10 },
    { "rank": 2, "username": "bob", "winCount": 7 },
    { "rank": 3, "username": "carol", "winCount": 5 }
  ]
}
```

上位 3 名。削除済みユーザー・master は除外。

6.9 管理者 API

いずれも JWT 必須 + role = master (AdminMiddleware)。

6.9.1 GET /api/admin/users

Query page, limit

items[]

項目 型 説明

id string ユーザー ID

username string ユーザー名

email string メール

項目 型 説明

role string user / master

winCount int 勝利数

deletedAt string | null 論理削除日時

createdAt string 登録日時

```
{
  "data": {
    "items": [
      {
        "id": "uuid",
        "username": "alice",
        "email": "a@example.com",
        "role": "user",
        "winCount": 3,
```

```
"deletedAt": null,  
"createdAt": "2026-01-01T00:00:00Z"  
}],  
"page": 1,  
"limit": 20,  
"total": 100  
}  
}
```

6.9.2 GET /api/admin/users/search

Query 内容

q username または email の部分一致（必須）

```
{  
  "data": [  
    {  
  
      "id": "uuid",  
      "username": "alice",  
      "email": "a@example.com",  
      "role": "user",  
      "winCount": 3  
    }  
  ]  
}
```

6.9.3 DELETE /api/admin/users/{id}

項目 内容

UseCase DeleteUserUseCase

Response 204

code HTTP 条件

user_in_active_game409 対戦中

cannot_delete_self 403 自分自身

cannot_delete_master403 master ユーザー

user_already_deleted409 削除済み

6.9.4 POST /api/admin/ranking/rebuild

項目 内容

UseCase RebuildRankingUseCase

Response 204

6.9.5 GET /api/admin/logs

Query | logType, userId, from, to, page, limit |

items[]

項目 型 説明

id string ログ ID

userId string | null ユーザー ID

logType string ログ種別

detail object JSONB

createdAt string ISO8601

{

"data": {

"items": [

{

"id": "uuid",

"userId": "uuid",

"logType": "guess",

"detail": { "gameId": "uuid", "turn": 3 },

"createdAt": "2026-06-18T12:00:00Z"

}],

"page": 1,

"limit": 20,

"total": 500

}

}

6.9.6 GET /api/admin/logs/download

Query logType, userId, from, to
Response Content-Type: text/csv
id,user_id,log_type,detail,created_at
uuid,uuid,guess,"{"gameld":"","..."}",2026-06-18T12:00:00Z

6.9.7 GET /api/admin/backup/status

```
{  
  "data": {  
    "lastSyncedAt": "2026-06-18T03:00:00Z",  
    "status": "ok"  
  }  
}
```

項目 型 説明

lastSyncedAt string 最終同期日時

status string ok / error

6.10 システム API6.10.1 GET /health

項目 内容

Controller SystemController

認証 不要

```
{  
  "status": "ok"  
}
```

6.11 DTO

UserDTO

属性 型 説明

ID string ユーザー ID

Username string ユーザー名

Role string user / master

WinCount int 累計勝利数

type UserDTO struct {

ID string

Username string

Role string

WinCount int

}

使用 : GET /api/me, POST /api/auth/register

GameStateDTO

属性 型 説明

GameID string ゲーム ID

Status string GameStatus

CurrentTurn int 現在ターン

CurrentTurnPlayerID string 現在ターンのプレイヤー ID

RemainingSeconds int 残り秒数

属性 型 説明

MyGuesses GuessSummaryDTO 自分の予想履歴

OpponentGuessCount int 相手の予想回数

type GameStateDTO struct {

GameID string

Status string

CurrentTurn int

CurrentTurnPlayerID string

RemainingSeconds int

MyGuesses []GuessSummaryDTO

OpponentGuessCount int

}

使用 : GET /api/games/{id}, WS GAME_STATE_SYNC

GuessSummaryDTO

属性 型 説明

Turn int ターン番号

GuessNumber string 予想 4 桁

DigitResults []int 桁判定結果

HitCount int 当たり数

IsAuto bool 自動送信フラグ

type GuessSummaryDTO struct {

Turn int

GuessNumber string

DigitResults []int
HitCount int
IsAuto bool
}
GuessResultDTO
属性 型 説明
GuessID string 予想 ID

属性 型 説明
PlayerID string 予想プレイヤー ID
DigitResults []int 桁判定結果
HitCount int 当たり数
IsWin bool 勝利確定か
NextTurnPlayerID string 次ターンのプレイヤー ID。勝利時は空文字
type GuessResultDTO struct {
GuessID string
PlayerID string
DigitResults []int
HitCount int
IsWin bool
NextTurnPlayerID string // 勝利時は空文字
}
使用 : WS GUESS_RESULT (HTTP では不使用)

6.12 Controller 責務

責務 内容

リクエスト受付 パス・クエリ・Body の取得
認証情報取得 JWT から userId / jti / role を context に設定
DTO 変換 Request → UseCase Input、Output → Response
UseCase 呼び出し 業務処理の委譲
エラー変換 UseCase error → HTTP status + error.code
Cookie 操作 login / refresh / logout の Set-Cookie
禁止 内容
DB / Redis 直接操作 UseCase 経由のみ
業務ロジック Controller に書かない

Repository 直接呼び出し 禁止

ゲーム判定 Domain 純粋関数のみ

Middleware の適用は §11 を参照する。

PDF との差分（意図的な補完）

項目 PDF 6.2 原文 本稿

POST /api/auth/refresh 6.1 にレート制限のみ 5.2 ・ 15.9 に基づき 6.3.3 として追加

Login Set-Cookie 6.2 に Body のみ Logout ・ Refresh 設計に合わせ 6.3.2 に追記

matching/start Response{ status: waiting } のみ PDF 準拠。 matched は status API / WS 参照

7. WebSocket

7.1 共通仕様7.1.1 エンドポイント

項目 内容

URL wss:///ws

プロトコル WebSocket （ TLS 必須）

許可オリジン 環境変数 WS_ALLOWED_ORIGINS（カンマ区切り）

実装 gorilla/websocket

7.1.2 メッセージ形式

Client → Server

項目 型 説明

type string イベント種別（必須）

（その他） — イベントごとのフィールドをトップレベルに配置

{ "type": "AUTH", "token": "jwt" }

Server → Client（通常）

項目 型 説明

type string イベント種別

data object ペイロード

{ "type": "AUTH_OK", "data": { "userId": "uuid" } }

例外

イベント 形式

PONG { "type": "PONG" }（data なし）

7.1.3 接続ライフサイクル

段階 ルール

接続直後 5 秒以内に AUTH 必須。未送信はサーバーが接続を切断

認証前 AUTH 以外のイベントはすべて拒否 (ERROR を返す)

認証後 ゲームイベント (SET_SECRET / GUESS / SYNC_REQUEST / PING) を受け付ける

多重接続 同一ユーザーの新接続が成功すると 旧接続を切断 (後勝ち)

切断時 ws_connection_logs.disconnected_at を更新、ws:user:{userId} を削除、対戦相手へ OPPONENT_STATUS

(connected: false)

ログアウト LogoutUseCase / DeleteUserUseCase / AutoLogoutUseCase 実行時に WS 接続を切断

7.1.4 認証 (AUTH)

HTTP API と同じ JWT 検証順序を適用する。

検証順 内容

1 署名

2 exp

3 jwt:revoked:{jti}

4 iat < force_logout_before

JWT Claims 内容

sub ユーザー ID

JWT Claims 内容

role user / master

jti JWT 識別子

iat 発行日時

exp 有効期限

条件 ERROR.code クライアント動作

exp 超過のみ token_expired POST /api/auth/refresh → 再接続 → AUTH 再送

署名不正・失効・強制ログアウト unauthorized トークン破棄 → /login へ

削除済みユーザー unauthorized 同上

再接続前の推奨フロー

POST /api/auth/refresh で最新 accessToken を取得

/ws に再接続（指数バックオフ、最大 5 回）
AUTH → 対戦中なら SYNC_REQUEST

7.1.5 通知タイミング

原則 内容

DB コミット後のみ Server → Client イベントは UseCase の DB コミット成功後に送信

保存先 ゲーム状態は PostgreSQL 。 WS は通知・同期専用

秘密情報 相手の秘密数字・相手の予想内容を WS で送らない

片方登録時 秘密数字を 1 人だけ登録した場合、Server → Client イベントは 送らない

7.1.6 アクティビティ更新

イベント 動作

PING 受信 users.last_activity_at を更新

ゲームイベント受信 同上

7.2 イベント一覧7.2.1 Client → Server

type UseCase 認証 概要

AUTH AuthenticateWebSocketUseCase 不要（接続直後） JWT 検証・接続登録

SET_SECRET SetSecretNumberUseCase 必須 秘密数字登録

GUESS SubmitGuessUseCase 必須 予想送信

SYNC_REQUEST SyncGameStateUseCase 必須 状態再同期要求

PING —（ハンドラ直接） 必須 キープアライブ

7.2.2 Server → Client

type 送信元 UseCase / 処理 タイミング

AUTH_OK AuthenticateWebSocketUseCase AUTH 成功直後

MATCHED StartMatchingUseCase（MatchPlayers 後） マッチング成立・DB コミット後

GAME_STATE_SYNC SyncGameStateUseCase / StartGameUseCase

/RecoverActiveGamesUseCase 同期要求・ゲーム開始・復元時

type 送信元 UseCase / 処理 タイミング

TURN_CHANGED StartGameUseCase / SubmitGuessUseCase /

HandleTimeoutUseCase /RecoverActiveGamesUseCase ターン開始時（初回含む）
GUESS_RESULT SubmitGuessUseCase / HandleTimeoutUseCase 予想処理完了・
DB コミット後
GAME_OVER SubmitGuessUseCase / HandleTimeoutUseCase
/CancelGameBySecretTimeoutUseCase ゲーム終了・ DB コミット後
OPPONENT_STATUS AuthenticateWebSocketUseCase / 切断処理 / LogoutUseCase
等 相手の接続状態変化時
ERROR 各 UseCase / ハンドラ 処理失敗・バリデーションエラー時
PONG ハンドラ直接 PING 受信時

7.3 接続・認証7.3.1 シーケンス（初回接続）

Client Server

```
| WS connect - |  
| AUTH {token} | AuthenticateWebSocketUseCase  
| AUTH_OK -----|  
| | Redis: ws:user:{userId} = connectionId  
| | ws_connection_logs INSERT  
| | 対戦中なら相手へ OPPONENT_STATUS connected:true
```

7.3.2 AUTH

項目 内容

UseCase AuthenticateWebSocketUseCase

ハンドラ WebSocketController

タイムアウト 接続後 5 秒以内

Request

項目 型 必須 説明

type string 必須 "AUTH"

token string 必須 JWT（Authorization ヘッダーは使用しない）

{ "type": "AUTH", "token": "jwt" }

Response — AUTH_OK

項目 型 説明

userId string 認証ユーザー ID（UUID）

{ "type": "AUTH_OK", "data": { "userId": "uuid" } }

処理概要

順 内容

- 1 JWT 検証
 - 2 ユーザー存在・未削除を確認
 - 3 同一ユーザーの旧 WS 接続があれば切断
 - 4 Hub に接続を登録
-

順 内容

- 5 Redis ws:user:{userId} に connectionId を保存（TTL JWT 残有効期限）
 - 6 ws_connection_logs に接続履歴を INSERT
 - 7 参加中ゲーム（WAITING_SECRET / IN_PROGRESS）の相手へ
OPPONENT_STATUS（connected: true）
 - 8 AUTH_OK を返す
- ERROR.code 条件
- token_expired JWT exp 超過
- unauthorized 署名不正・失効・強制ログアウト・ユーザー不存在・削除済み
- validation_errortoken 欠落

7.4 Client → Server イベント詳細7.4.1 SET_SECRET

項目 内容

UseCase SetSecretNumberUseCase

前提 ゲーム WAITING_SECRET、参加者、自分の秘密数字未登録

Request

項目 型 必須 条件

type string 必須 "SET_SECRET"

gameId string 必須 UUID

secretNumberstring 必須 4 桁・0～9・重複なし

{ "type": "SET_SECRET", "gameId": "uuid", "secretNumber": "1234" }

成功時の Server → Client

状況 送信イベント

片方のみ登録完了 なし（相手の登録状態は通知しない）

両者登録完了 StartGameUseCase 経由で GAME_STATE_SYNC →
TURN_CHANGED

ERROR.code

code 条件

validation_error 二重登録

invalid_digit_length 4 桁以外

invalid_digit 数字以外
duplicate_digit 桁重複
not_found ゲーム不存在
forbidden 参加者でない
game_already_started IN_PROGRESS (秘密数字登録不可)
game_already_finished 終了済み
rate_limit_exceeded Redis ロック取得失敗
unauthorized 未認証
internal_error DB 更新失敗

7.4.1 SET_SECRET

順 内容

- 1 Redis キー game:{gameId}:player:{userId}:secret_lock を Lua で acquire
- 2 NewSecretNumber 検証
- 3 SecretHashService.Hash
- 4 平文破棄
- 5 games FOR UPDATE → ハッシュ保存
- 6 両者完了 → StartGameUseCase

7.4.2 GUESS

項目 内容

UseCase SubmitGuessUseCase

前提 ゲーム IN_PROGRESS、参加者、自分のターン

Request

項目 型 必須 条件

type string 必須 "GUESS"

gameId string 必須 UUID

guessNumber string 必須 4 桁・0～9・重複なし

{ "type": "GUESS", "gameId": "uuid", "guessNumber": "5678" }

成功時の Server → Client (送信順)

順 イベント 条件

1 GUESS_RESULT 常に両プレイヤーへ

2 GAME_OVER isWin: true の場合

2 TURN_CHANGED isWin: false の場合

処理概要（ UseCase 内・クライアントには非公開）

順 内容

1 Redis キー game:{gameId}:player:{userId}:guess_lock を Lua で acquire

2 games FOR UPDATE

3 相手 secret ハッシュ 取得

4 SecretHashService.Verify → digitResults

5 Domain.IsWin

6 guesses INSERT FinishGameService

7 COMMIT 後 WS 通知

ERROR.code

code 条件

not_your_turn 自分のターンでない

game_not_started WAITING_SECRET

game_already_finished FINISHED

invalid_digit_length / invalid_digit / duplicate_digit 入力不正

not_found ゲーム不存在

forbidden 参加者でない

rate_limit_exceeded 連打

validation_error 相手秘密数字未登録

code 条件

unauthorized 未認証

internal_error DB 更新失敗

7.4.3 SYNC_REQUEST

項目 内容

UseCase SyncGameStateUseCase

用途 再接続・ページ再読み込み後の状態復元

Request

項目 型 必須 説明

type string 必須 "SYNC_REQUEST"

gameId string 必須 UUID

{ "type": "SYNC_REQUEST", "gameId": "uuid" }

成功時

GAME_STATE_SYNC を要求者に 1 件送信。ペイロードは GET /api/games/{id} のレ

スポンスと同一（ GameStateDTO ）。

ERROR.code

code 条件

not_found ゲーム不存在

forbidden 参加者でない

unauthorized 未認証

7.4.4 PING

項目 内容

UseCase なし

用途 キープアライブ・セッション維持

Request

```
{ "type": "PING" }
```

Response

```
{ "type": "PONG" }
```

8. エラーコード

Code HTTP 内容

validation_error 400 入力不正

invalid_digit_length 400 4 桁以外

duplicate_digit 400 重複数字

invalid_digit 400 数字以外

unauthorized 401 JWT 無効・期限切れ・失効

token_expired 404 アクセストークンの有効期限切れ。クライアントは

/api/auth/refresh でトークン

を更新してからリトライする

forbidden 403 権限不足

not_found 404 リソースなし

Code HTTP 内容

not_your_turn 409 自分のターンでない

game_not_started 409 ゲームが未開始（ WAITING_SECRET ）

game_already_finished 409 ゲームが終了済み

duplicate_user 409 ユーザー名 / メール重複

user_in_active_game 409 対戦中（マッチング不可・削除不可）
already_in_matching 409 既にマッチング待機中
user_already_deleted 409 削除済み
cannot_delete_self 403 自分は削除不可
cannot_delete_master 403 master は削除不可
rate_limit_exceeded 429 連打防止・API レート制限
internal_error 500 サーバー内部エラー

9. PostgreSQL9.1 概要

項目 内容

エンジン PostgreSQL 15

ORM GORM

テスト SQLite（テスト環境のみ）

バックアップ 別インスタンスの PostgreSQL（差分 UPSERT 同期）

接続 環境変数 DATABASE_URL

設計方針

原則 内容

正本 ゲーム状態の真実値は常に PostgreSQL

論理削除 ユーザーは物理削除せず deleted_at で論理削除

トランザクション ゲーム更新は SELECT ... FOR UPDATE で行ロック

二重参加防止 アプリ層チェックが主。DB は Partial Index で検索を高速化

9.2 テーブル一覧

テーブル 用途

users ユーザーアカウント

games 対戦ゲーム

guesses 予想履歴

match_histories 勝敗履歴（guess_win 終了時のみ）

rankings ランキング Read Model

matching_queue マッチング待機キュー

activity_logs 操作ログ

login_logs ログイン・ログアウト履歴

ws_connection_logs WebSocket 接続履歴

9.3 users

カラム 型 制約 説明

id UUID PK ユーザー ID

username VARCHAR50 UNIQUE NOT NULL 3 ～ 50 文字、英数字と _

email VARCHAR255 UNIQUE NOT NULL RFC5322 準拠

password VARCHAR255 NOT NULL bcrypt ハッシュ (cost 12)

role VARCHAR20 NOT NULL user / master

win_count INT NOT NULL DEFAULT 0 累計勝利数

deleted_at TIMESTAMPTZ NULL 論理削除日時

deleted_by UUID NULL 削除した管理者 ID

last_activity_at TIMESTAMPTZ NOT NULL 最終操作日時

created_at TIMESTAMPTZ NOT NULL 作成日時

updated_at TIMESTAMPTZ NOT NULL 更新日時

備考

削除済みユーザー (deleted_at IS NOT NULL) はログイン・マッチング対象外

master は対戦・マッチング不可

9.4 games

カラム 型 制約 説明

id UUID PK ゲーム ID

status VARCHAR20 NOT NULL WAITING_SECRET / IN_PROGRESS / FINISHED

player1_id UUID NOT NULL FK → users 先手プレイヤー

player2_id UUID NOT NULL FK → users 後手プレイヤー

player1_secret VARCHAR255 NULL player1 の位置別 HMAC 連結。未登録 NULL

player2_secret VARCHAR255 NULL player2 同上

current_turn_player_id UUID NULL FK → users 現在ターンのプレイヤー ID

current_turn INT NOT NULL DEFAULT 1 現在ターン (1 始まり)

winner_id UUID NULL FK → users 勝者 ID

started_at TIMESTAMPTZ NULL ゲーム開始日時

finished_at TIMESTAMPTZ NULL ゲーム終了日時

created_at TIMESTAMPTZ NOT NULL 作成日時

updated_at TIMESTAMPTZ NOT NULL 更新日時

status の意味

status 意味

WAITING_SECRET 秘密数字登録待ち

IN_PROGRESS 対戦中

FINISHED 終了（勝敗確定または secret_setup_timeout）

ゲーム終了時

FINISHED 遷移後、current_turn_player_id は NULL にクリアする

secret_setup_timeout 時は winner_id = NULL

9.5 進行中ゲームの二重参加防止

DB レベルでは Partial Index で検索を高速化する。一意性は保証しない。

Partial Index

CREATE INDEX idx_games_active_players

ON games (player1_id, player2_id)

WHERE status IN ('WAITING_SECRET', 'IN_PROGRESS');

アプリ層チェック

UseCase チェック内容

StartMatchingUseCase status IN ('WAITING_SECRET', 'IN_PROGRESS') のゲームに参加中なら user_in_active_game

DeleteUserUseCase 同上

競合時

SELECT ... FOR UPDATE により先にロックを取得した処理のみが進行する。

9.6 guesses

カラム 型 制約 説明

id UUID PK 予想 ID

game_id UUID NOT NULL FK → games ゲーム ID

player_id UUID NOT NULL FK → users 予想プレイヤー ID

turn INT NOT NULL ゲーム全体の通しターン番号

guess_number VARCHAR4 NOT NULL 予想 4 桁

digit_results JSONB NOT NULL 桁判定結果 [0,1,0,0] 形式

hit_count INT NOT NULL 当たり数（0～4）

is_auto BOOLEAN NOT NULL DEFAULT false タイムアウト自動予想か

created_at TIMESTAMPTZ NOT NULL 作成日時

制約

UNIQUE (game_id, turn, player_id)

備考

turn は games.current_turn のスナップショット

1 ターンにつき手番プレイヤーは 1 人のみのため、同一 turn で player 違いの複数行は通常発生しない

9.7 match_histories

カラム 型 制約 説明

id UUID PK 履歴 ID

game_id UUID UNIQUE ゲーム ID

winner_id UUID NOT NULL 勝者 ID

loser_id UUID NOT NULL 敗者 ID

winner_username VARCHAR50 — 勝者名 (スナップショット)

loser_username VARCHAR50 — 敗者名 (スナップショット)

finished_at TIMESTAMPTZ — 終了日時

作成条件

条件 作成

guess_win で終了 作成する (FinishGameUseCase)

secret_setup_timeout 作成しない

9.8 rankings

項目 内容

性質 Read Model (CQRS)。対戦 TX からは更新しない

更新タイミング RankingRebuildWorker (RANKING_REBUILD_CRON、 §16) および
POST /api/admin/ranking/rebuild

更新処理 RebuildRankingUseCase が users.win_count から全件再集計

除外 削除済みユーザー、 master

updated_at バッチ実行完了時刻。 React の「最終更新」表示に使用

9.9 matching_queue

項目 内容

ペアリング StartMatchingUseCase が同一 TX 内で MatchPlayersUseCase を呼ぶ

並び順 created_at ASC、先頭 = player1

削除 成立時に 2 行 DELETE

Worker 使用しない

9.10 activity_logs

カラム 型 説明

id UUID PK ログ ID

user_id UUID NULL 操作ユーザー ID

log_type VARCHAR50 ログ種別

detail JSONB 詳細 (JSON)

created_at TIMESTAMPTZ 作成日時

log_type 例

log_type 内容

guess 予想実行

timeout タイムアウト自動予想

game_start ゲーム開始

game_over ゲーム終了

user_delete ユーザー削除

保持ポリシー

項目 値

保持日数 ACTIVITY_LOG_RETENTION_DAYS 日 (既定 90 日)

削除 LogRetentionWorker

9.11 login_logs

カラム 型 説明

id UUID PK ログ ID

user_id UUID ユーザー ID

action VARCHAR20 login / logout

created_at TIMESTAMPTZ 作成日時

保持ポリシー

項目 値

保持日数 LOGIN_LOG_RETENTION_DAYS 日 (既定 90)

削除 LogRetentionWorker

9.12 ws_connection_logs

カラム 型 説明

id UUID PK ログ ID

user_id UUID ユーザー ID
connection_id VARCHAR64 接続 ID
connected_at TIMESTAMPTZ 接続日時
disconnected_at TIMESTAMPTZ NULL 切断日時
保持ポリシー
項目 値
保持日数 WS_LOG_RETENTION_DAYS 日（既定 30）
削除 LogRetentionWorker

9.13 インデックス

テーブル インデックス
users email, username, deleted_at
games player1_id, player2_id, status、 Partial Index （ 9.5 参照）
guesses (game_id, turn)
match_histories winner_id, loser_id
rankings rank
activity_logs (log_type, created_at)
matching_queue (status, created_at)
バックアップ DB 全テーブル updated_at（差分 UPSERT 用・必須）

9.14 バックアップ同期対象

BackupWorker が差分 UPSERT するテーブル。

テーブル 同期

users ○

games ○

guesses ○

match_histories ○

rankings ○

activity_logs ○

login_logs ○

ws_connection_logs×

matching_queue ×

同期方式

Redis backup:status.lastSyncedAt を基準に updated_at > lastSyncedAt の行のみ

UPSERT

初回 (lastSyncedAt 未設定) は全件対象

10. Redis10.1 概要

項目 内容

クライアント go-redis v9

接続 環境変数 REDIS_URL

用途 連打防止・ターン期限・JWT 失効・WS 接続管理・バックアップ状態

設計方針

原則 内容

正本にしない Redis 喪失時は DB から復元可能な設計とする

短命データ ロック・失効キーは TTL 付き

ターン期限 IN_PROGRESS ゲームの残り秒数算出に使用

10.2 連打防止

ルール 内容

サフィックス 連打防止ロックは _lock で終える

プレフィックス禁止 キー先頭に lock: を付けない

ゲームスコープ game:{gameId}:player:{playerId}:...

管理スコープ admin:{adminId}:...

同一キー 手動 GUESS と HandleTimeoutUseCase は同一 guess_lock を使う

操作 キー TTL

予想 game:{gameId}:player:{playerId}:guess_lock GAME_LOCK_SECONDS 秒

秘密数字 game:{gameId}:player:{playerId}:secret_lock GAME_LOCK_SECONDS 秒

ランキング再集計 admin:{adminId}:ranking_rebuild_lock ADMIN_LOCK_SECONDS
秒

ログ DL admin:{adminId}:log_download_lock ADMIN_LOCK_SECONDS 秒

ユーザー削除 admin:{adminId}:user_delete_lock ADMIN_LOCK_SECONDS 秒

ロック取得失敗時

層 code

HTTP / WS rate_limit_exceeded

備考

予想とタイムアウト自動予想は同一 guess_lock キーを使用

HandleTimeoutUseCase はロック取得失敗時、何もせず正常終了

10.3 ターン期限

キー

game:{gameId}:turn

値 (JSON)

```
{  
  "turn": 5,  
  "playerId": "uuid",  
  "startedAt": "20260618T120000Z",  
  "expiresAt": "20260618T120030Z"  
}
```

フィールド 説明

turn 現在ターン番号

playerId 手番プレイヤー ID

startedAt ターン開始日時

expiresAt ターン期限 (TurnTimeoutWorker の判定基準)

登録タイミング

UseCase タイミング

StartGameUseCase ゲーム開始・1 ターン目

SubmitGuessUseCase 未勝利時の次ターン

HandleTimeoutUseCase 未勝利時の次ターン

RecoverActiveGamesUseCase サーバー起動時の復元

削除タイミング

UseCase 条件

SubmitGuessUseCase / HandleTimeoutUseCase 予想処理後

FinishGameUseCase ゲーム終了時

CancelGameBySecretTimeoutUseCase 秘密数字期限切れ終了時
Worker

TurnTimeoutWorker が expiresAt < now() のキーを走査

間隔: TURN_TIMEOUT_POLL_SECONDS 秒 (既定 1 秒)

10.4 認証補助

キー 値 TTL

jwt:revoked:{jti} 1 JWT 残有効期限

user:{userId}:force_logout_before Unix 秒 30 日（自動延長なし）
ws:user:{userId} connectionId JWT 有効期限
jwt:revoked:{jti}
ログアウト時に SET
JWT 検証時に存在チェック
user:{userId}:force_logout_before
AutoLogoutUseCase が無操作ユーザーに SET
JWT の iat がこの値より古い場合は拒否
再ログイン後は新 JWT の iat で自然に無視可能
ws:user:{userId}
WS 認証成功時に SET
ログアウト・強制ログアウト・削除時に DELETE
同一ユーザーの新接続時、旧接続を切断（後勝ち）

10.5 バックアップ状態

キー

backup:status

値（JSON）

```
{  
  "lastSyncedAt": "20260618T030000Z",  
  "status": "ok"  
}
```

フィールド 説明

lastSyncedAt 最終同期日時

status ok / error

更新

BackupWorker 成功時 : lastSyncedAt = now, status = ok

失敗時 : status = error（3 回リトライ）

10.6 JWT 検証（Redis 連携）

JWT Claims 内容

sub ユーザー ID

role user / master

jti JWT 識別子

iat 発行日時
exp 有効期限
検証順
順 検証
1 署名
2 exp
3 jwt:revoked:{jti}
4 iat < force_logout_before

10.7 障害時の扱い

データ 喪失時の対応
game:{gameId}:turn DB の IN_PROGRESS を読み、TURN_DURATION_SECONDS
秒でターン再作成
ロック / JWT 失効 無視（短命のため許容）
ws:user:{userId} 再接続で再登録
backup:status 初回同期として全件 UPSERT

11. Middleware11.1 Middleware 一覧

Middleware 対象 処理
CORSMiddleware 全 API CORS_ALLOWED_ORIGINS のオリジンのみ許可

Middleware 対象 処理
RecoverMiddleware 全 API panic → 500 internal_error
RequestLogMiddleware 全 API activity_logs 記録（パスワード・JWT は除外）
AuthMiddleware JWT 必須 APIJWT 検証、userId を context に設定
AdminMiddleware /api/admin/* role = master 検証
ActivityUpdateMiddlewareJWT 必須 APIusers.last_activity_at 更新
RateLimitMiddleware login / registerIP 単位レート制限

11.2 CORSMiddleware

項目 内容
設定 環境変数 CORS_ALLOWED_ORIGINS（カンマ区切り）
動作 許可オリジン以外からのリクエストを拒否

11.3 RecoverMiddleware

項目 内容

動作 panic を捕捉し HTTP 500 を返す

レスポンス { "error": { "code": "internal_error", ... } }

ログ サーバー側にスタックトレースを記録

11.4 RequestLogMiddleware

項目 内容

記録先 activity_logs

除外 パスワード、JWT 本体

内容 リクエストパス、ユーザー ID、ステータス等

11.5 AuthMiddleware

項目 内容

対象 Cookie `access_token` 必須の API

検証 署名 → exp → jwt:revoked:{jti} → iat < force_logout_before

成功 userId, jti, role を context に設定

失敗 401 unauthorized

適用例

GET /api/me

POST /api/matching/start

GET /api/games/{id}

管理 API （ AdminMiddleware と併用）

11.6 AdminMiddleware

項目 内容

対象 /api/admin/*

項目 内容

条件 JWT の role = master

失敗 403 forbidden

11.7 ActivityUpdateMiddleware

項目 内容
対象 JWT 必須 API
動作 `users.last_activity_at = now()`
用途 `AutoLogoutUseCase` の無操作判定
WebSocket
PING 受信時も同等の更新を行う

11.8 RateLimitMiddleware

対象 制限
POST `/api/auth/login` 同一 IP 10 回 / 分
POST `/api/auth/register` 同一 IP 5 回 / 分
その他認証必須 API 同一ユーザー 120 回 / 分
超過時
HTTP 429 `rate_limit_exceeded`

11.9 Middleware 適用順

CORSMiddleware
→ RecoverMiddleware
→ RequestLogMiddleware
→ RateLimitMiddleware (`login/register` のみ)
→ AuthMiddleware (認証必須 API)
→ AdminMiddleware (`/api/admin/*` のみ)
→ ActivityUpdateMiddleware (認証必須 API)
→ Controller

12. バックグラウンド処理12.1 Worker 一覧

Worker 間隔 担当
TurnTimeoutWorker `TURN_TIMEOUT_POLL_SECONDS` `HandleTimeoutUseCase`
SecretSetupTimeoutWorker `SECRET_TIMEOUT_POLL_SECONDS`
CancelGameBySecretTimeoutUseCase
AutoLogoutWorker `AUTO_LOGOUT_POLL_SECONDS` `AutoLogoutUseCase`
RankingRebuildWorker `RANKING_REBUILD_CRON` `RebuildRankingUseCase`
LogRetentionWorker `LOG_RETENTION_CRON` `LogRetentionUseCase`
BackupWorker `BACKUP_CRON` バックアップ同期

RefreshTokenCleanupWorkerREFRESH_TOKEN_CLEANUP_CRON refresh_tokens 削除

12.3 TurnTimeoutWorker

項目 内容

間隔 TURN_TIMEOUT_POLL_SECONDS 秒（既定 1 秒）

UseCase HandleTimeoutUseCase

検出方法

Redis game:{gameId}:turn の expiresAt < now() を走査

処理概要

順 内容

1 期限切れターンを検出

2 gameId, playerId, now を HandleTimeoutUseCase に渡す

3 自動予想 (isAuto=true) → 通常予想と同ルールで判定

12.4 SecretSetupTimeoutWorker

項目 内容

間隔 SECRET_TIMEOUT_POLL_SECONDS 秒（既定 1 秒）

UseCase CancelGameBySecretTimeoutUseCase

条件

status = WAITING_SECRET

created_at + SECRET_SETUP_SECONDS < now

結果

FINISHED（勝者なし）

WS GAME_OVER (reason: secret_setup_timeout)

match_histories は作成しない

12.5 AutoLogoutWorker

項目 内容

間隔 AUTO_LOGOUT_POLL_SECONDS 秒（既定 60 秒）

UseCase AutoLogoutUseCase

条件

last_activity_at < now - SESSION_TIMEOUT_MINUTES

処理

順 内容

1 user:{userId}:force_logout_before = now を SET

2 WS 接続があれば切断し ERROR(unauthorized)

3 login_logs に logout (reason: auto_logout)

12.6 RankingRebuildWorker

項目 内容

スケジュール RANKING_REBUILD_CRON (§16 、既定 /10 * * * UTC)

項目 内容

UseCase RebuildRankingUseCase

処理

users.win_count から rankings を上書き再集計

削除済みユーザー・master を除外

12.7 LogRetentionWorker

項目 内容

スケジュール LOG_RETENTION_CRON

UseCase LogRetentionUseCase

削除対象

テーブル 条件 バッチサイズ

activity_logs created_at < now - ACTIVITY_LOG_RETENTION_DAYS

RETENTION_BATCH_SIZE

login_logs created_at < now - LOGIN_LOG_RETENTION_DAYS

RETENTION_BATCH_SIZE

ws_connection_logsconnected_at < now - WS_LOG_RETENTION_DAYS

RETENTION_BATCH_SIZE

方針

バッチ DELETE + バッチ間スリープ（長時間ロック回避）

削除失敗は警告ログに記録し次回に持ち越し

12.8 BackupWorker

項目 内容

スケジュール BACKUP_CRON

方式 差分 UPSERT

処理概要

順 内容

1 Redis backup:status.lastSyncedAt 取得（未設定なら epoch 0）

2 対象テーブルから updated_at > lastSyncedAt の行を取得

3 バックアップ DB へ主キー単位 UPSERT

4 成功時 backup:status に lastSyncedAt=now, status=ok

5 失敗時 status=error、3 回リトライ

対象テーブル

users, games, guesses, match_histories, rankings, activity_logs, login_logs

対象外

ws_connection_logs, matching_queue

RPO

最大 24 時間（日次同期）

12.9 サーバー起動順

main() 起動順：

1. DB 接続

2. Redis 接続3. WebSocketHub 起動

4. RecoverActiveGamesUseCase 実行5. 各 Worker 起動

6. HTTP サーバー listen

Worker 起動に MatchPlayersWorker は含めない。

RecoverActiveGamesUseCase（起動時 1 回）

Worker 起動に MatchPlayersWorker は含めない。

対象 処理

IN_PROGRESS ゲーム Redis ターン情報を復元または再作成

WAITING_SECRET かつ期限切れ CancelGameBySecretTimeoutUseCase

12.10 障害復旧

データ 喪失時の対応

PostgreSQL バックアップ DB からリストア。RPO ≤ 24 時間

Redis turn DB の IN_PROGRESS を読み、TURN_DURATION_SECONDS 秒でターン再開

Redis ロック / 失効 無視（短命のため許容）

Redis backup:status全件 UPSERT として初回同期

13. セキュリティ13.1 基本方針

原則 内容

サーバー ゲーム状態・権限・判定結果は常にサーバー（PostgreSQL UseCase）を正とする

最小権限 user / master の権限を厳密に分離する

秘密の非開示 秘密数字・パスワード・JWT・リフレッシュトークンをログ・API・WS に載せない

コミット後通知 WebSocket 通知は DB コミット成功後のみ送信する

監査可能性 activity_logs / login_logs で操作を追跡できるようにする

変更管理 セキュリティ関連の設定変更時は本章とデプロイ手順書を更新する

13.2 セキュリティ目標

目標 内容

認証の確実性 なりすまし・セッション乗っ取り・トークン再利用を防ぐ

認可の厳格性 参加者以外・master 以外の操作を拒否する

ゲーム公平性 桁判定・勝敗を Domain 純粋関数のみで行う

可用性 連打・レート超過・競合状態を Redis ロックで制御する

データ保護 バックアップ DB を含め games（秘密数字含む）の取り扱いを本番同等にする

保守性 チェックリスト・初動手順・Worker 監視を運用必須とする

13.3 サーバー側必須検証

クライアント送信値を信頼しない。以下は すべてサーバー側で検証 する。

13.3.1 検証一覧

検証項目 適用箇所 失敗時 code (代表)

JWT 署名 AuthMiddleware、AuthenticateWebSocketUseCase unauthorized

JWT exp 超過のみ AuthMiddleware、AuthenticateWebSocketUseCase
token_expired

JWT 失効 (jti) AuthMiddleware、AuthenticateWebSocketUseCase
unauthorized

強制ログアウト AuthMiddleware、AuthenticateWebSocketUseCase unauthorized

削除済みユーザー LoginUseCase、AuthMiddleware、WS AUTH unauthorized

Role (master) AdminMiddleware、StartMatchingUseCase forbidden

ゲーム参加者 GetGameStateUseCase、SetSecretNumberUseCase、
SubmitGuessUseCase、SyncGameStateUseCase forbidden

ゲーム status SetSecretNumberUseCase、SubmitGuessUseCase
game_not_started /game_already_finished

自ターン SubmitGuessUseCase not_your_turn

4 桁・0 ~ 9・重複なし SetSecretNumberUseCase、SubmitGuessUseCase
invalid_digit_* / duplicate_digit

マッチング前提 StartMatchingUseCase user_in_active_game /already_in_matching

リフレッシュトークン RefreshTokenUseCase unauthorized

token_expired と unauthorized の使い分け

条件 HTTP WebSocket ERROR.code

exp 超過のみ token_expired (404) token_expired

署名不正・失効・強制ログアウト unauthorized (401) unauthorized

リフレッシュ不可の失効 unauthorized (401) —

クライアントは token_expired 受信時に POST /api/auth/refresh で accessToken
を更新してからリトライする。refreshも失敗した場合は /login へ遷移する。

13.3.2 検証

層 担当

Middleware JWT 検証、Role、CORS、レート制限、panic 回復

UseCase 参加者・ターン・status・入力・DB 整合・refresh_tokens 照合

Domain 桁判定・勝敗 (純粋関数、副作用なし)

Worker 期限切れ・自動ログアウト・ログ削除・バックアップ・refresh クリーンアッ
プ

層 禁止すること

Domain DB / Redis / WebSocket アクセス、副作用

UseCase HTTP レスポンス直接生成、Domain 外のゲーム判定
Controller / WS ハンドラ 業務ロジック、Repository 直接呼び出し、桁判定
Infrastructure ゲームルール判定、ターン進行決定
Router UseCase 直接呼び出し、DB / Redis 直接操作

13.4 認証・セッション管理13.4.1 アクセストークン（JWT）

項目 内容

方式 Bearer JWT

項目 内容

ヘッダー Cookie `access_token` (`credentials: include`)

署名鍵 環境変数 `JWT_SECRET` (32 文字以上)

有効期限 `JWT_EXPIRY_MINUTES` (既定 60 分)

Claims `sub`, `role`, `jti`, `iat`, `exp`

検証順

順 検証

1 署名

2 `exp`

3 `jwt:revoked:{jti}`

4 `iat < user:{userId}:force_logout_before`

失効登録 (`LogoutUseCase`)

項目 内容

キー `jwt:revoked:{jti}`

値 1

TTL JWT 残有効期限

13.4.2 リフレッシュトークン

項目 内容

生成 `crypto/rand` 64 バイト → hex エンコード (128 文字)

保存 `refresh_tokens.token_hash` = SHA256 (平文トークン)

運搬 `HttpOnly` Cookie `refresh_token` (Request Body 不使用)

Path `/api/auth/refresh`

Secure `true`

SameSite `Strict`

有効期限 `REFRESH_TOKEN_EXPIRY_DAYS` 日 (既定 7 日)

更新 API POST /api/auth/refresh (RefreshTokenUseCase)

レート制限 同一 IP 30 回 / 分

RefreshTokenUseCase のセキュリティ要件

項目 内容

ローテーション 更新成功時、旧トークンを revoked にし新トークンを INSERT

family_id 同一ログインセッションのトークン群で family_id を共有

再使用検出 revoked トークンの再使用時、同一 family_id の active をすべて revoked

再使用時の初動 ws:user:{userId} 削除・WS 切断・unauthorized

LogoutUseCase対象ユーザーの active refresh_tokens をすべて revoked

RequestLogMiddleware / activity_logs で記録禁止

refresh_token Cookie 本体

JWT 本体

パスワード (平文・ハッシュ)

13.4.3 ログアウト・自動ログアウト

UseCase 処理

LogoutUseCase jwt:revoked:{jti} SET 、 refresh_tokens revoked 、 ws:user 削除、 login_logs 、 Cookie 削除

AutoLogoutUseCaseforce_logout_before SET 、 WS 切断、 login_logs (reason: auto_logout)

DeleteUserUseCase対象ユーザーの WS 切断、 matching_queue 削除

項目 内容

自動ログアウト条件 last_activity_at < now - SESSION_TIMEOUT_MINUTES

既定 5 分無操作

Worker AutoLogoutWorker (1 分間隔)

更新契機 JWT 必須 API 、 WS PING 、 WS ゲームイベント

13.4.4 WebSocket 接続セキュリティ

項目 内容

プロトコル WSS (TLS 必須)

接続後 5 秒以内に AUTH 必須

認証前 AUTH 以外はすべて ERROR

多重接続 後勝ち (旧接続切断)

オリジン WS_ALLOWED_ORIGINS

接続記録 ws_connection_logs （ 30 日保持、バックアップ対象外）
再接続 POST /api/auth/refresh → 再接続 → AUTH → 対戦中なら SYNC_REQUEST

13.5 認可・権限13.5.1 権限マトリクス

機能 user master

ログイン・登録 ○ ○

マッチング・対戦（ HTTP / WS ） ○ ×

ランキング閲覧 ○ ○

プロフィール・履歴閲覧 ○ ○

POST /api/matching/start ○ × （ forbidden ）

WS SET_SECRET / GUESS ○ × （ forbidden ）

ユーザー管理・ログ・ CSV × ○

ランキング再集計 × ○

バックアップ状況確認 × ○

13.5.2 master の制約

操作 結果

POST /api/matching/startforbidden

WS SET_SECRET / GUESSforbidden

WS 接続（ AUTH ） 可（管理・監視用途）

自分自身の削除 cannot_delete_self

master ユーザーの削除 cannot_delete_master

13.5.3 ゲームスコープ認可

操作 追加条件

GET /api/games/{id} player1_id または player2_id

WS SET_SECRET / GUESS / SYNC_REQUEST同上

他人の gameId forbidden または not_found

13.6 秘密情報の保護

13.6.1 秘密数字（ player1_secret / player2_secret ）

項目 内容

pepper GAME_SECRET_PEPPER (§16 必須。 32 バイト以上)

保存 UseCase 経由で SecretHashService.Hash 生成後、 games.player*_secret にハッシュのみ保存

平文禁止 PostgreSQL ・ バックアップ DB ・ ログに平文 4 桁を保存しない

運用上の必須事項

項目 要求

DB アクセス 本番・バックアップ DB とも最小権限の接続ユーザー

バックアップ games は同期対象。バックアップ DB も本番同等のアクセス制御

ダンプ 本番データのダンプを開発環境へ持ち込まない

手動 UPDATE ゲーム進行中の games 手動 UPDATE 禁止 (調査は READ のみ)

13.6.2 パスワード

項目 内容

保存 bcrypt (cost = 12)

登録 RegisterUserUseCase でハッシュ化後 INSERT

照合 LoginUseCase で bcrypt 比較

ログ 平文・ハッシュとも記録禁止

13.6.3 アクセストークン・リフレッシュトークン

項目 内容

accessToken メモリ保持推奨。 localStorage 永続化は推奨しない

refresh_token HttpOnly Cookie のみ。 JavaScript から読み取らない

ログ JWT 本体・ refresh_token 本体を RequestLog に出力しない

13.7 通信セキュリティ

項目 内容

HTTP HTTPS (TLS 必須)

WebSocket WSS (TLS 必須)

CORS CORS_ALLOWED_ORIGINS のみ許可

WS オリジン WS_ALLOWED_ORIGINS のみ許可

Cookie refresh_token は HttpOnly / Secure / SameSite=Strict

本番デプロイ前チェック

項目

- 1 HTTP / WS とも TLS 終端が有効
 - 2 許可オリジンがワイルドカード * になっていない
-

項目

- 3 JWT_SECRET がデフォルト値・短い文字列でない
- 4 REFRESH_TOKEN Cookie が Secure=true (本番 HTTPS 前提)

13.8 入力検証・ゲーム公平性13.8.1 入力ルール

対象 ルール

secretNumber / guessNumber 4 桁、0 ~ 9、重複なし

username 3 ~ 50 文字、3+\$

email (register) RFC5322 準拠、255 文字以下

email (login) メール形式、50 文字以下

password 8 文字以上

13.8.2 乱数

用途 方式

タイムアウト自動予想 crypto/rand (重複なし 4 桁)

リフレッシュトークン crypto/rand 64 バイト

禁止 math/rand

13.8.3 競合制御

競合 解決

手動 GUESS vs HandleTimeout 同一 game:{gameId}:player:{playerId}:guess_lock。

先着のみ進行

ゲーム更新 SELECT ... FOR UPDATE

後着の手動 GUESS not_your_turn または game_not_started / game_already_finished

後着の HandleTimeout 正常終了 (何もしない)

ゲーム status 返却 code (代表)

WAITING_SECRET game_not_started

FINISHED game_already_finished

自ターンでない not_your_turn

13.9 ログ・監査・記録13.9.1 記録してよいもの

種別 内容

login_logs login / logout （ auto_logout 含む）

activity_logs guess, timeout, game_start, game_over, user_delete 等

ws_connection_logs connectionId, connected_at, disconnected_at

RequestLog パス、 userId 、 HTTP ステータス

13.9.2 記録してはいけないもの

種別 理由

パスワード（平文・ハッシュ） 認証情報漏洩

秘密の数字 ゲーム公平性

種別 理由

JWT 本体 セッション乗っ取り

refresh_token 本体 セッション乗っ取り

13.9.3 CSV 出力（ DownloadActivityLogsUseCase ）

項目 内容

カラム id, user_id, log_type, detail, created_at

detail 秘密数字・ JWT ・ refresh_token を含めない

ロック admin:{adminId}:log_download_lock （ ADMIN_LOCK_SECONDS ）

13.9.4 保持・削除

テーブル 保持（既定） 削除 Worker

activity_logs ACTIVITY_LOG_RETENTION_DAYS （ 90 日） LogRetentionWorker

login_logs LOGIN_LOG_RETENTION_DAYS （ 90 日） 同上

ws_connection_logs WS_LOG_RETENTION_DAYS （ 30 日） 同上

refresh_tokens expires_at / revoked 経過分 RefreshTokenCleanupWorker

監査上の注意

ws_connection_logs はバックアップ対象外。 30 日超の WS 接続調査は不可
削除失敗は警告ログに記録し次回 Worker に持ち越す（アプリ停止しない）

13.10 レート制限・ Redis13.10.1 HTTP レート制限

対象 制限

POST /api/auth/login 同一 IP 10 回 / 分

POST /api/auth/register 同一 IP 5 回 / 分

POST /api/auth/refresh 同一 IP 30 回 / 分

その他認証必須 API 同一ユーザー 120 回 / 分

超過 : rate_limit_exceeded (429)

13.10.2 Redis

操作 キー TTL

GUESS / タイムアウト game:{gameId}:player:

{playerId}:guess_lock GAME_LOCK_SECONDS

SET_SECRET game:{gameId}:player:{playerId}:secret_lock GAME_LOCK_SECONDS

ランキング再集計 admin:{adminId}:ranking_rebuild_lock ADMIN_LOCK_SECONDS

ログ DL admin:{adminId}:log_download_lock ADMIN_LOCK_SECONDS

ユーザー削除 admin:{adminId}:user_delete_lock ADMIN_LOCK_SECONDS

13.10.3 運用監視

監視項目 閾値例 対応

login / refresh rate_limit_exceeded 急増 通常の 10 倍以上 ブルートフォース疑い → IP 調査

unauthorized 急増 通常の 10 倍以上 JWT_SECRET 漏洩・設定ミス疑い

RefreshToken family 一括失効ログ 1 件でも トークン盗用疑い → ユーザー通知検討

BackupWorker status=error 1 回連続 RPO 超過リスク → 即エスカレーション

監視項目 閾値例 対応

internal_error 率 1% 超 サーバー異常調査

13.11 データ保護・バックアップ

項目 内容

保存先 PostgreSQL

方式 差分 UPSERT (updated_at > lastSyncedAt)

間隔 BACKUP_CRON

RPO 最大 24 時間

状態管理 Redis backup:status

同期対象テーブル

users, games, guesses, match_histories, rankings, activity_logs, login_logs

同期対象外

ws_connection_logs, matching_queue

セキュリティ要件

項目 要求

BACKUP_DATABASE_URL 本番 DB と別ネットワークセグメント推奨

games 含有 秘密数字を含むため、バックアップ媒体の取り扱いを本番 DB と同等に
リストアテスト 四半期 1 回以上実施し手順書を更新

失敗時 status=error、最大 3 回リトライ

13.12 環境変数・シークレット管理13.12.1 機密変数

変数 用途 運用ルール

JWT_SECRET JWT 署名 32 文字以上。ローテーション手順を文書化

DATABASE_URL 本番 DB Git / ログ / CI 出力禁止

BACKUP_DATABASE_URL バックアップDB 同上

REDIS_URL Redis 同上

NUMDUEL_MASTER_EMAIL master seed 初回のみ。リポジトリに含めない

NUMDUEL_MASTER_PASSWORD master seed 同上。強力なパスワード必須

GAME_SECRET_PEPPER 秘密数字 HMAC32 バイト以上。本番・ステージング・開発
で同一値を使い回さない。Git / ログ禁止

13.12.2 CI/CD

項目 内容

シークレット保存 GitHub Secrets

イメージ GHCR

本番値 CI ログ・PR コメントに出力しない

13.12.3 変更管理

以下を変更した場合：

本章またはデプロイ手順書を更新

ステージングで login / refresh / logout / WS / 自動ログアウトを確認

本番反映

変更対象 確認項目

GAME_SECRET_PEPPER 変更後は既存 secret ハッシュが無効になるため、原則ローテーション不可。漏洩時は新規ゲームのみ有効化する手順を文書化

SECRET_SETUP_SECONDS SecretSetupTimeoutWorker ・

CancelGameBySecretTimeout の判定

TURN_DURATION_SECONDS /TURN_TIMEOUT_POLL_SECONDS ターン期限・タイムアウト Worker

ACTIVITY_LOG_RETENTION_DAYS /LOGIN_LOG_RETENTION_DAYS

/WS_LOG_RETENTION_DAYS LogRetentionWorker 削除対象

RANKING_REBUILD_CRON RankingRebuildWorker

13.13 マスター初期化

項目 内容

タイミング 初回起動時のみ

条件 users に master が 0 件

入力 NUMDUEL_MASTER_EMAIL, NUMDUEL_MASTER_PASSWORD

スキップ master が 1 件でも存在すれば seed しない

運用ルール

項目 要求

初回パスワード 初回ログイン後に変更推奨

master 数 1 件を推奨

削除 cannot_delete_master により API から削除不可

監査 master による admin 操作は activity_logs に記録される

13.14 侵害・障害時の初動

事象 初動

JWT_SECRET 漏洩疑い 鍵ローテーション、全ユーザー force_logout_before 更新、refresh_tokens 一括revoked を検討

refresh_token 盗用疑い（family 一括失効） activity_logs で userId 特定、該当ユーザーへのパスワード変更推奨

DB 漏洩疑い パスワードは bcrypt。 games（秘密数字）・email の影響範囲を調査
不正 admin 操作 activity_logs で userId ・ logType 検索

Redis 全喪失 RecoverActiveGamesUseCase、ロック・失効は許容

BackupWorker 連続失敗 status=error を監視、RPO 超過リスクをエスカレーション
ログ調査手順

順 内容

- 1 activity_logs を logType / userId / 期間で絞り込み
 - 2 login_logs でセッション確立・終了を確認
 - 3 ws_connection_logs (30 日以内) で接続履歴を確認
 - 4 秘密数字・JWT・refresh_token は調査出力に含めない
-

13.15 禁止事項

禁止 内容

- クライアント判定 秘密数字の照合・勝敗判定をフロントで行わない
- 平文パスワード保存 DB に平文 password を保存しない
- 秘密の外部送信 WS / HTTP / ログ / CSV に秘密数字を含めない
- トークンのログ出力 JWT 本体・refresh_token 本体を RequestLog に書かない
- refresh Body 送信 リフレッシュトークンを Request Body で送らない
- Domain 外判定 桁判定を Controller / WS ハンドラ / Redis で行わない
- COMMIT 前 WS 通知 DB コミット前に Server → Client を送らない
- 本番 DB 直操作 ゲーム進行中の手動 UPDATE 禁止
- master の対戦 master によるマッチング・SET_SECRET / GUESS 禁止
- 仕様外 code 8 章にない error.code を返さない
- デフォルト秘密鍵 JWT_SECRET 未設定・弱い値での本番起動禁止
- math/rand タイムアウト予想・トークン生成に使用禁止
- 開発環境への本番ダンプ 匿名化なしで games を含むダンプを持ち込まない

13.16 レイヤ別セキュリティ責務

層 セキュリティ責務

- Router Middleware チェーン適用、TLS 終端（リバースプロキシ連携）
- Middleware 認証・認可・CORS・レート制限・RequestLog（機密除外）
- Controller DTO 変換、UseCase 呼び出し、error → code 変換
- UseCase ビジネスルール検証、トランザクション、Redis / WS 連携
- Domain 純粋関数による判定（副作用なし）
- Infrastructure DB / Redis I/O のみ。ルール判定禁止
- Worker 期限・ログ削除・バックアップ・自動ログアウト・refresh クリーンアップ

14. 画面仕様14.1 基本方針

原則 内容

サーバー正本 ゲーム状態・勝敗・ターンはサーバー応答を優先する

判定禁止 秘密数字の照合・勝敗判定をフロントで行わない

秘密非表示 相手の秘密数字・相手の予想内容を表示しない

二重送信防止 ボタン内スピナー + 送信中は disabled

認証 accessToken はメモリ保持。refresh_token は HttpOnly Cookie

master 対戦・マッチング画面は利用不可（API が forbidden を返す）

14.2 ルーティング

パス 画面 Page コンポーネント 認証

/register 新規登録 RegisterPage 不要

/login ログイン LoginPage 不要

/matching マッチング MatchingPage 必須（user）

/game/:id 対戦 GamePage 必須（user）

パス 画面 Page コンポーネント 認証

/ranking ランキング RankingPage 必須

/profile プロフィール ProfilePage 必須

/admin 管理画面 AdminPage 必須（master のみ）

リダイレクト

条件 動作

未認証で保護ページアクセス /login へ

ログイン済みで /login または /register アクセス /matching へ（master は /admin）

role=master で /matching または /game/:id アクセス /admin へ（または forbidden トースト）

401 unauthorized（リフレッシュ不可）user 状態クリア（setUser(null））→ /login

403 forbidden（master が対戦操作）トースト表示、操作キャンセル

14.3 認証・セッション（画面共通） 14.3.1 初期化

順 処理

1 アプリ起動時、メモリに accessToken があれば GET /api/me で有効性確認

2 token_expired（404）なら POST /api/auth/refresh を試行

3 refresh 成功 → refresh 成功（Set-Cookie で access_token 更新）を保持、
/api/me 再実行

4 refresh 失敗 → user 状態クリア（setUser(null)）、未認証状態

14.3.2 API リクエスト共通

項目 内容

ヘッダー Cookie `access_token` (`credentials: include`)

credentials refresh 用 Cookie 送信のため include (fetch) または withCredentials (axios)

token_expired (404) POST /api/auth/refresh → 成功時に元リクエストを 1 回リトライ

unauthorized (401) user 状態クリア (setUser(null)) → /login

429 rate_limit_exceeded トースト 「操作が多すぎます。しばらく待ってください。」

14.3.3 WebSocket 共通 (useWebSocket)

項目 内容

URL wss:///ws

接続前 POST /api/auth/refresh で refresh 成功 (Set-Cookie で access_token 更新) を取得

接続後 5 秒以内 AUTH (Body: { "type": "AUTH" } のみ)

キープアライブ 30 秒間隔で PING → PONG

token_expired (WS) refresh → 再接続 → AUTH 再送

unauthorized (WS) user 状態クリア (setUser(null)) → /login

切断時 指数バックオフ再接続 (最大 5 回)

14.4 新規登録 (/register)

項目 内容

Page RegisterPage

API POST /api/auth/register (RegisterUserUseCase)

項目 内容

認証 不要

入力項目

項目 型 必須 クライアント検証

username string 必須 3 ~ 50 文字、4+\$

email string 必須 メール形式、255 文字以下

password string 必須 8 文字以上

passwordConfirmstring 必須 password と一致

操作

操作 動作

送信 POST /api/auth/register

成功（201） トースト「登録完了」 → /login へ遷移

validation_error フォーム上部に error.message

duplicate_user フォーム上部に error.message

rate_limit_exceeded トースト表示

リンク

ラベル 遷移先

ログインはこちら /login

14.5 ログイン（/login）

項目 内容

Page LoginPage

API POST /api/auth/login（LoginUseCase）

認証 不要

入力項目

項目 型 必須 クライアント検証

email string 必須 メール形式、50 文字以下

password string 必須 8 文字以上

操作

操作 動作

送信 POST /api/auth/login

成功（200） user 情報を React state に保持（JWT 文字列は保持しない）。

refresh_token は Set-Cookie（自動）

role=user /matching へ遷移

role=master /admin へ遷移

unauthorized フォーム上部「メールまたはパスワードが正しくありません」

validation_error フォーム上部に error.message

リンク

ラベル 遷移先

新規登録はこちら /register

14.6 マッチング（/matching）

項目 内容

Page MatchingPage

対象 role=user のみ

WebSocket 接続・AUTH 必須（MATCHED 受信用）

表示

状態 表示

idle 「マッチングを開始できます」

waiting スピナー + 「対戦相手を探しています ...」

matched 遷移中（/game/:id へ）

操作

ボタン API 動作

マッチング開始 POST /api/matching/startstatus=waiting 表示。即時ペアリング成功時も HTTP は waiting

キャンセル POST /api/matching/cancelstatus=cancelled → idle 表示

状態取得

手段 内容

GET /api/matching/statusidle / waiting / matched。matched 時 gameId 取得

WS MATCHED data.gameId 受信 → /game/:gameId へ遷移

MATCHED 受信時

順 動作

1 data.gameId を取得

2 /game/:gameId へ navigate

3 GamePage で WS 接続（未接続なら）・SYNC_REQUEST または GET /api/games/{id}

ヘッダー

リンク 遷移先

ランキング /ranking

プロフィール /profile

ログアウト POST /api/auth/logout → user 状態クリア（setUser(null)） → /login

エラー

code 表示

user_in_active_game トースト + 進行中 gameId があれば /game/:id へ誘導

already_in_matching 「既に待機中です」

forbidden master はこの画面を表示しない

14.7 対戦（ /game/:id ）

項目 内容

Page GamePage

状態管理 useGameState （ Context + useReducer ）

初期データ GET /api/games/{id} （ GetGameStateUseCase ）

リアルタイム WebSocket （ SET_SECRET / GUESS / SYNC_REQUEST / PING ）

14.7.1 画面ロード

順 処理

1 URL から gameId 取得

2 WS 未接続なら接続 → AUTH

3 GET /api/games/{id} で GameStateDTO 取得

4 status=FINISHED なら結果モーダル表示

5 以降 WS イベントで状態更新

14.7.2 秘密数字フェーズ（ WAITING_SECRET ）

項目 内容

入力 4 桁数字 + 送信ボタン

WS 送信 SET_SECRET { gameId, secretNumber }

残り秒数 remainingSeconds （ API ） または SECRET_SETUP_SECONDS カウントダウン

相手状態 「相手の登録待ち」のみ（相手が登録済みかは表示しない）

自秘密登録済み 入力欄 disabled + 「登録済み」

WS イベント

イベント 画面更新

GAME_STATE_SYNC 全状態更新。IN_PROGRESS へ遷移

TURN_CHANGED 対戦フェーズへ移行、タイマーリセット

GAME_OVER （ secret_setup_timeout ） 結果モーダル

ERROR トースト （ code に応じた message ）

14.7.3 対戦フェーズ（ IN_PROGRESS ）

表示

項目 データソース

残り秒数 remainingSeconds (GAME_STATE_SYNC / TURN_CHANGED)
現在ターン currentTurnPlayerID と自分の userId 比較
自分の予想履歴 myGuesses[] 。 digitResults を ○ / × 表示 (1 ○ , 0)
相手の予想回数 opponentGuessCount のみ (内容非公開)
自ターン 入力欄 active
相手ターン 入力欄 disabled + 「相手のターンです」
操作
操作 WS 送信
予想送信 GUESS { gameId, guessNumber }
WS イベント
イベント 画面更新
TURN_CHANGED タイマーリセット、ターン表示更新
GUESS_RESULT 履歴追加 (digitResults, hitCount, isAuto)
GAME_OVER 結果モーダル
OPPONENT_STATUSconnected=false → 切断バナー。 true → バナー解除
GUESS_RESULT 表示ルール

項目 内容

digitResults 両プレイヤーに同内容を表示
相手の guessNumber表示しない (サーバーが送らない)
isAuto=true 履歴に「自動」ラベル

14.7.4 結果モーダル (GAME_OVER)

data.reason 表示
guess_win 自分が winner なら「勝利!」、それ以外「敗北 ...」
secret_setup_timeout 「登録時間切れのためゲーム終了」
操作 動作
閉じる /matching へ遷移

14.7.5 切断・再接続

状態 表示
切断検知 バナー「再接続中 ...」
再接続成功 バナー「接続が復旧しました」(3 秒で消去)
再接続フロー refresh → WS 再接続 → AUTH → SYNC_REQUEST
再接続パラメータ

項目 値

バックオフ 指数 (1s, 2s, 4s, 8s, 16s)

最大試行 5 回

失敗時 トースト + /matching へ誘導

14.7.6 対戦画面エラー

code 表示・動作

not_found トースト → /matching

forbidden トースト → /matching

not_your_turn トースト

game_not_started トースト

game_already_finished 結果モーダルまたは /matching

invalid_digit_* / duplicate_digit入力欄下にエラー

rate_limit_exceeded トースト

14.8 ランキング (/ranking)

項目 内容

Page RankingPage

API GET /api/ranking (GetRankingUseCase)

認証 JWT 必須

表示

項目 内容

テーブル 上位 3 名 (rank, username, winCount)

空 「ランキングデータがありません」

操作

操作 動作

ページ表示 マウント時に API 取得

再取得 手動リロードボタン (任意) またはページ再訪問

14.9 プロフィール (/profile)

項目 内容

Page ProfilePage

認証 JWT 必須

ヘッダー表示 (GET /api/me/profile)

項目 内容

username ユーザー名

winCount 累計勝利数

rank 順位。圏外は「圏外」

タブ

タブ API 表示

勝敗履歴 GET /api/me/match-history?page&limitgameId, winnerUsername, loserUsername, finishedAt

ログイン履歴 GET /api/me/login-history?page&limitaction, createdAt

WS 接続履歴 GET /api/me/ws-history?page&limit connectionId, connectedAt, disconnectedAt

ページング

パラメータ 既定 上限

page 1 —

limit 20 100

備考

secret_setup_timeout で終了したゲームは勝敗履歴に含まれない

login_logs 保持 90 日、 ws_connection_logs 保持 30 日

14.10 管理画面 (/admin)

項目 内容

Page AdminPage

対象 role=master のみ

ガード GET /api/me で role 確認。 user なら /matching へ

14.10.1 ユーザー管理タブ

操作 API 動作

一覧 GET /api/admin/users?page&limit テーブル表示

検索 GET /api/admin/users/search?q=部分一致結果

削除 DELETE /api/admin/users/{id} 確認ダイアログ → 204

削除エラー

code 表示

user_in_active_game トースト

cannot_delete_self トースト
cannot_delete_master トースト
user_already_deleted トースト

14.10.2 ログタブ

操作 API 動作
検索 GET /api/admin/logs?logType&userId&from&to&page&limit テーブル表示
CSV DL GET /api/admin/logs/download?... ファイルダウンロード

14.10.3 ランキングタブ

操作 API 動作
再集計 POST /api/admin/ranking/rebuild204 → トースト「再集計しました」

14.10.4 バックアップタブ

操作 API 表示
状況確認 GET /api/admin/backup/statuslastSyncedAt, status (ok / error)

14.11 共通 UI

14.11.1 トースト

項目 内容
位置 画面上部
自動消去 3 秒
用途 成功・エラー・警告

14.11.2 ローディング

項目 内容
ボタン スピナー + ラベル「処理中 ...」
二重送信 送信中は disabled
ページ 初回ロード時はスケルトンまたはスピナー

14.11.3 認証エラー処理

条件 動作
HTTP 401 unauthorized user 状態クリア (setUser(null)) → /login
HTTP 404 token_expired POST /api/auth/refresh → 成功時リトライ / 失敗時 /login

WS ERROR unauthorizeduser 状態クリア (setUser(null)) → /login
WS ERROR token_expiredrefresh → 再接続 → AUTH

14.11.4 ログアウト

操作 動作

ヘッダーボタン POST /api/auth/logout

成功 user 状態クリア (setUser(null))、WS 切断、/login へ

14.11.5 自動ログアウト

条件 動作

SESSION_TIMEOUT_MINUTES 無操作 AutoLogoutWorker
(AUTO_LOGOUT_POLL_SECONDS) が切断

14.12 画面 × API × WebSocket 対応表

画面 HTTP API WebSocket (受信) WebSocket (送信)

RegisterPage POST /api/auth/register — —

LoginPage POST /api/auth/login — —

MatchingPage POST /api/matching/start, cancel, GET /api/matching/status,
POST /api/auth/logout MATCHED AUTH, PING

GamePage GET /api/games/{id}

GAME_STATE_SYNC, TURN_CHANGED, GUESS_RESULT, GAME_OVER,
OPPONENT_STATUS, ERROR

AUTH, SET_SECRET, GUESS, SYNC_REQUEST, PING

RankingPage GET /api/ranking — —

ProfilePage GET /api/me/profile, match-history, login-history, ws-history — —

AdminPage GET/DELETE /api/admin/*, POST /api/admin/ranking/rebuild — —

共通 GET /api/me, POST /api/auth/refresh ERROR (全画面) AUTH, PING

14.13 フロントエンド構成

frontend/src/

pages/

RegisterPage.tsx

LoginPage.tsx

MatchingPage.tsx
GamePage.tsx
RankingPage.tsx
ProfilePage.tsx
AdminPage.tsx
hooks/
useAuth.tsx # accessToken, login, logout, refresh
useWebSocket.tsx # 接続, AUTH
useGameState.tsx # 対戦画面 reducer (GamePage のみ)
api/
client.ts # fetch ラッパー, token_expired リトライ
types/
dto.ts # GameStateDTO, GuessResultDTO 等
router/
AppRouter.tsx # ルート定義, 認証ガード

16. 環境変数

変数	説明	既定値
PORT	HTTP listen	8090
REDIS_ADDR	Redis (REDIS_URL 未設定時)	—
APP_ENV	production 時 Redis 必須	—
COOKIE_SECURE	Cookie Secure	false (本番 true)
SKIP_MIGRATE	1 で AutoMigrate スキップ	—
VITE_API_BASE_URL	フロント本番 API	—
VITE_WS_BASE_URL	フロント本番 WS	—

16.4 Docker Compose

サービス	ホストポート
postgres	5434
postgres-backup	5433
redis	6379

サービス	ホストポート
backend	8090

14.13 フロントエンド構成

```
frontend/src/
├─ api/client.ts
├─ components/ (game, admin, layout, ui)
├─ constants/navigation.ts
├─ hooks/ (useAuth, useWebSocket, useGameState, useGamePage, useAdminPage, useAsyncAction, useLogout, useToast)
├─ lib/ (apiBase, format, labels, routes, validation)
├─ pages/ (Register, Login, Matching, Game, Ranking, Profile, Admin)
├─ router/ (AppRouter.tsx, guards.tsx)
└─ types/dto.ts
```

14.14 ルートガード

コンポーネント	条件	リダイレクト
GuestOnly	未認証	認証済み → role に応じ遷移
RequireAuth	認証必須	未認証 → <code>/login</code>
RequireUser	role=user	master → <code>/admin</code>
RequireMaster	role=master	user → <code>/matching</code>

14.15 本番 API / WS URL

環境	設定
ローカル	Vite proxy <code>/api</code> , <code>/ws</code> → <code>localhost:8090</code>
Vercel	<code>VITE_API_BASE_URL</code> , <code>VITE_WS_BASE_URL</code>

シナリオ

M2 A のみ start queue 1 件、games なし、MATCHED なし

M3 start 中 DB 失敗 queue INSERT 含め ROLLBACK

M4 Worker 経由ペアリング 存在しない

18.1 テスト種別一覧

種別 対象 実行 合格基準

Domain 単体 JudgeDigits, IsWin, NewSecretNumber, Game go test

./internal/domain/... カバレッジ 100% 以上

UseCase 結合 主要 UseCase + Repository (SQLite/ mock Redis) go test

./internal/usecase/... 主要パス全通過

API 結合 全 HTTP エンドポイント go test ./internal/controller/... または

router テスト 正常 / 異常すべて定義どおり

WS 結合 AUTH → SET_SECRET → GUESS → GAME_OVER backend WS テスト 一連シナリオ通過

競合テスト タイムアウト vs 手動 GUESS 同時 UseCase + Redis 実装 二重進行なし

フロント単体 hooks / pages (Vitest) npm test 主要コンポーネント通過

E2E 2 ユーザー対戦シナリオ CI (docker compose 起動後) 毎回実行・通過

18.3 テスト環境

項目 開発 / CI 本番

DB (backend テスト) SQLite (インメモリ可) —

DB (E2E) PostgreSQL (compose) —

Redis 実 Redis または miniredis —

認証 テスト用 JWT_SECRET (32 文字以上) —

フロント Vitest + Testing Library —

アサーション testify (Go) —

E2E 起動構成

docker compose で backend / frontend / postgres / redis を起動し、2 ユーザー分の HTTP WS クライアントからシナリオを実行する。

18.4 Domain 単体テスト

対象ファイル: internal/domain/judge.go, game.go, guess.go 等

18.4.1 JudgeDigits

秘密数字 予想 期待 digitResults期待 hitCount

1 1234 1234 1,1,1,1 4
2 1234 5678 0,0,0,0 0
3 1234 1456 0,1,0,0 1
4 1234 4321 0,0,0,0 0 （位置不一致は外れ）

18.4.2 IsWinhitCount 期待

2 3 false
3 0 false

18.4.3 NewSecretNumber / バリデーション入力 期待

1 1234 成功
2 123 invalid_digit_length
3 12345 invalid_digit_length
4 12a4 invalid_digit
5 1123 duplicate_digit

18.4.4 Game エンティティ操作 期待

1 SetSecret （ player1 ） player1_secret 設定、 status は WAITING_SECRET
2 両者 SetSecret 後 StartGamestatus = IN_PROGRESS, current_turn = 1, 先手 = player1
3 AddGuess （未勝利） current_turn 1、ターン交代
4 Finish （ hitCount=4 ） status = FINISHED, winner_id 設定

18.5 UseCase 結合テスト

Repository は SQLite 、 Redis は miniredis または test double を使用する。

18.5.1 認証系

UseCase 必須シナリオ

RegisterUserUseCase 正常登録 / duplicate_user / validation_error

LoginUseCase 正常 login （Set-Cookie access_token + refresh_token） /

unauthorized

RefreshTokenUseCase 正常ローテーション / revoked 再使用 → family 一括失効 / unauthorized

LogoutUseCase jwt:revoked 登録 / refresh revoked / Cookie 削除

18.5.2 マッチング系

UseCase 必須シナリオ

StartMatchingUseCase 1 人待機 / 2 人で即マッチ（同一 TX 内 MatchPlayers） / already_in_matching / user_in_active_game / master → forbidden

MatchPlayersUseCase 2 人ペアリング / 1 人以下は no-op / player1 = 先着

CancelMatchingUseCase 待機取消

18.5.3 ゲーム系

UseCase 必須シナリオ

SetSecretNumberUseCase 正常登録 / 両者完了 → StartGame 呼び出し / game_not_started

SubmitGuessUseCase 正常予想 / not_your_turn / 勝利時 FinishGame 同一 TX / match_histories + win_count

HandleTimeoutUseCase 期限切れ自動予想 / 手動先行時は no-op

CancelGameBySecretTimeoutUseCase WAITING_SECRET 期限切れ → FINISHED（勝者なし）

FinishGame winner 確定時のみ / 同一 TX 内完結

SyncGameStateUseCase 参加者のみ取得 / forbidden

SubmitGuessUseCase 勝利 GUESS で match_histories INSERT 失敗 → guesses 含め ROLLBACK

UseCase 必須シナリオ

HandleTimeoutUseCase 勝利時 win_count UPDATE 失敗 → games FINISHED も ROLLBACK

18.5.4 管理・その他

UseCase 必須シナリオ

DeleteUserUseCase 正常削除 / cannot_delete_master / cannot_delete_self / user_in_active_game

RebuildRankingUseCase win_count 再集計 / master 除外
AutoLogoutUseCase last_activity_at 超過 → force_logout_before
LogRetentionUseCase 保持期限超過分のみバッチ DELETE
RecoverActiveGamesUseCaseIN_PROGRESS ターン復元
合格基準: 上記 主要パスすべて通過 (表に列挙した異常系を含む)

18.6 API 結合テスト

全エンドポイントについて 正常 1 件 + 代表異常 1 件以上 をテストする。
メソッド パス 正常 代表異常
POST /api/auth/register 201 duplicate_user, validation_error
POST /api/auth/login 200 Cookie unauthorized
POST /api/auth/refresh 200 + 新 Cookie unauthorized (Cookie なし)
POST /api/auth/logout 204 unauthorized
GET /api/me 200 unauthorized, token_expired
POST /api/matching/start 200 / 待機 already_in_matching, forbidden (master)
POST /api/matching/cancel 200 unauthorized
GET /api/matching/status 200 unauthorized
GET /api/games/{id} 200 forbidden (非参加者) , not_found
GET /api/me/profile 200 unauthorized
GET /api/me/match-history 200 unauthorized
GET /api/me/login-history 200 unauthorized
GET /api/me/ws-history 200 unauthorized
GET /api/ranking 200 unauthorized
GET /api/admin/users 200 forbidden (user)
GET /api/admin/users/search 200 forbidden
DELETE /api/admin/users/{id} 200 cannot_delete_master
POST /api/admin/ranking/rebuild 200 forbidden
GET /api/admin/logs 200 forbidden
GET /api/admin/logs/download 200 CSV forbidden, rate_limit_exceeded
GET /api/admin/backup/status 200 forbidden
GET /health 200 —
token_expired 検証
条件 HTTP
JWT exp 超過のみ 404 token_expired
署名不正・失効 401 unauthorized

18.7 WebSocket 結合テスト

18.7.1 基本フロー

順 Client → Server Server → Client 確認

1 接続 — —

2 AUTH (Body: { "type": "AUTH" } のみ。token は Cookie から取得) AUTH_OK 5 秒以内

3 SET_SECRET GAME_STATE_SYNC 秘密数字非含有

4 (相手 SET_SECRET) TURN_CHANGED player1 先手

5 GUESS GUESS_RESULT digitResults のみ (秘密非含有)

6 (勝利 GUESS) GAME_OVER match_histories 作成

18.7.2 異常・再接続シナリオ 期待

1 期限切れ JWT で AUTHERROR token_expired

2 失効 JWT で AUTH ERROR unauthorized

3 非参加者が GUESS ERROR forbidden

4 相手ターンで GUESS ERROR not_your_turn

5 SYNC_REQUEST GAME_STATE_SYNC

6 PING PONG + last_activity_at 更新

7 マッチング成立 MATCHED (両者)

18.7.3 通知順序確認

1 GUESS_RESULT / GAME_OVER / TURN_CHANGED は DB コミット後にのみ送信

2 WS ペイロードに player1_secret / player2_secret が含まれない

18.8 競合テストシナリオ 期待

1 同一ターンで HandleTimeout と SubmitGuess が同時 先着 1 件のみ処理。後着は no-op または rate_limit_exceeded / not_your_turn

2 同一 guess_lock で二重 GUESS 2 件目は rate_limit_exceeded

3 同一 secret_lock で二重 SET_SECRET 2 件目は rate_limit_exceeded

- 4 タイムアウト処理中に DB 上ターンが移動済み HandleTimeout は正常終了（変更なし）
 - 5 StartMatching 同期マッチング中の二重 startalready_in_matching
 - 6 勝利 GUESS 中に FinishGameService がerror DB に 何も残らない
 - 7 COMMIT 前 GAME_OVER 送信されない
- 合格基準: ゲーム状態・guesses 件数に 二重進行がない

18.9 フロントエンド単体テスト

実行: npm test (Vitest + Testing Library)

対象 必須シナリオ

useAuth login 成功で user 保持 / GET /api/me セッション確認 / refresh リトライ / logout でクリア

useWebSocketAUTH 送信 / token_expired 時 refresh → 再接続

useGameStateGUESS_RESULT で reducer 更新 / 秘密数字を state に持たない

対象 必須シナリオ

AppRouter 未認証 → /login / master → /admin ガード

RegisterPage validation エラー表示

LoginPage unauthorized メッセージ表示

合格基準: 上記テストスイート すべて通過

18.10 E2E テスト

実行: CI 毎回 (docker compose 起動 → シナリオ)

18.10.1 シナリオ : 2 ユーザー対戦

順 操作 確認

1 userA / userB を register → login両者 login 成功 (Cookie セッション確立)

2 userA が matching/start 待機

3 userB が matching/start 両者 MATCHED 、 gameId 取得

4 両者 WS 接続 + AUTH AUTH_OK

5 両者 SET_SECRET GAME_STATE_SYNC

6 先手が GUESS (外れ) GUESS_RESULT TURN_CHANGED

7 後手が GUESS (当たり 4 桁) GAME_OVER 、勝者確定

- 8 GET /api/ranking 勝者 winCount 1
- 9 GET /api/me/match-history 履歴 1 件

18.10.2 シナリオ : 認証フロー

順 操作 確認

- 1 login → refresh (Cookie) refresh 成功 (Set-Cookie で access_token 更新)
- 2 logout 204、refresh 不可
- 3 GET /api/me (ログアウト後) unauthorized

18.10.3 シナリオ : 管理 API

順 操作 確認

- 1 master login role=master
 - 2 GET /api/admin/users 200
 - 3 user JWT で GET /api/admin/usersforbidden
- 合格基準: 18.10.1 ~ 18.10.3 すべて通過

18.11 セキュリティ関連テスト項目 期待

- 1 refresh_token を Request Body で送る 使用しない (Cookie のみ)
- 2 RequestLog / activity_logs パスワード・JWT 本体・refresh_token 非含有
- 3 GET /api/games/{id} レスポンス 秘密数字非含有
- 4 GUESS_RESULT 秘密数字非含有
- 5 revoked refresh 再使用 同一 family_id 一括失効
- 6 master が matching/start forbidden
- 7 CSV ダウンロード detail に秘密数字非含有

17. ディレクトリ構成

```
backend/  
├─ main.go / go.mod / Dockerfile  
├─ config/ controller/ crypto/ db/ dto/ middleware/  
├─ migrations/migrate.go  
├─ model/ repository/ redis/ router/ testutil/  
└─ usecase/ websocket/ worker/
```

```
NumDuel/  
├─ .env.example / docker-compose.yml  
├─ .github/workflows/cicd.yml  
├─ k6/scenarios/health.js  
└─ backend/ / frontend/
```

18. テスト18.1 フロント単体テスト18.2 CI 実行順序

frontend-ci → backend-ci → integration-test → container-build → security-scan
→ publish-images → production-migrate → smoke-test → k6-load → deploy-
production

19. CI/CD パイプライン19.1 基本方針

原則	内容
workflow	<code>.github/workflows/cicd.yml</code>
PR	CI のみ
push (main/master/develop)	CI + migrate + smoke + k6 + Vercel

19.3 ジョブ一覧

順	job id	push 時のみ
1	frontend-ci	
2	backend-ci	
3	integration-test	
4	container-build	
5	security-scan	
6	publish-images	○
7	production-migrate	○
8	smoke-test	○
9	k6-load	○
10	deploy-production	○

19.4 ジョブ詳細

- **frontend-ci:** Node 22, `npm ci` → `npm test` → `npm run build`
- **backend-ci:** Go 1.25, `go vet` → `go test` → `go build`
- **integration-test:** Postgres + Redis services, migrate, `/health`
- **security-scan:** gosec, npm audit, Trivy, gitleaks
- **deploy-production:** Vercel CLI + 本番 migrate

19.5 Secrets

Secret	用途
VERCEL_TOKEN / VERCEL_ORG_ID / VERCEL_PROJECT_ID	Vercel deploy
PRODUCTION_DATABASE_URL	本番 migrate
PRODUCTION_API_BASE_URL	smoke / k6

19.6 Environments

`production` (Required reviewers 推奨)

20. 本番デプロイ

コンポーネント	ホスティング
フロント	Vercel (Root: <code>frontend</code>)
API / WS	別サービス
DB / Redis	マネージド

20.2 Vercel 設定

Build: `npm run build` / Output: `dist` / Env: `VITE_API_BASE_URL`, `VITE_WS_BASE_URL`

20.3 バックエンド本番要件

APP_ENV=production, REDIS 必須, COOKIE_SECURE=true, CORS/WS origins に Vercel ドメイン

20.4 デプロイ順

publish-images → production-migrate → smoke-test → k6-load → deploy-production

20.5 Cookie 認証（本番）

Cookie	Path	Secure
access_token	/	true
refresh_token	/api/auth/refresh	true

JWT は JavaScript から参照しない。 `credentials: include` で API / WS を呼び出す。